



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Creación de *RAG*
Documentación Técnica



Presentado por Lydia Blanco Ruiz
en Universidad de Burgos — 13 de mayo
de 2026

Tutor: Álgvar Arnaiz González y César Ignacio
García Osorio

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	32
Apéndice B Especificación de Requisitos	37
B.1. Introducción	37
B.2. Objetivos generales	37
B.3. Catálogo de requisitos	37
B.4. Especificación de requisitos	41
Apéndice C Especificación de diseño	65
C.1. Introducción	65
C.2. Diseño de datos	65
C.3. Diseño arquitectónico	66
C.4. Diseño procedimental	76
C.5. Diseño de interfaces	76
Apéndice D Documentación técnica de programación	89
D.1. Introducción	89
D.2. Estructura de directorios	89

D.3. Manual del programador	95
D.4. Compilación, instalación y ejecución del proyecto	106
D.5. Pruebas del sistema	109
Apéndice E Documentación de usuario	111
E.1. Introducción	111
E.2. Requisitos de usuarios	111
E.3. Instalación	111
E.4. Manual del usuario	111
Apéndice F Anexo de sostenibilización curricular	113
F.1. Introducción	113
Bibliografía	115

Índice de figuras

A.1. <i>Burndown chart</i> del <i>sprint</i> 1.	4
A.2. <i>Burndown chart</i> del <i>sprint</i> 2.	5
A.3. <i>Burndown chart</i> del <i>sprint</i> 3.	6
A.4. <i>Burndown chart</i> del <i>sprint</i> 4.	7
A.5. <i>Burndown chart</i> del <i>sprint</i> 5.	8
A.6. <i>Burndown chart</i> del <i>sprint</i> 6.	9
A.7. <i>Burndown chart</i> del <i>sprint</i> 7.	10
A.8. <i>Burndown chart</i> del <i>sprint</i> 8.	11
A.9. <i>Burndown chart</i> del <i>sprint</i> 9.	13
A.10. <i>Burndown chart</i> del <i>sprint</i> 10.	14
A.11. <i>Burndown chart</i> del <i>sprint</i> 11.	15
A.12. <i>Burndown chart</i> del <i>sprint</i> 12.	16
A.13. <i>Burndown chart</i> del <i>sprint</i> 13.	17
A.14. <i>Burndown chart</i> del <i>sprint</i> 14.	19
A.15. <i>Burndown chart</i> del <i>sprint</i> 15.	20
A.16. <i>Burndown chart</i> del <i>sprint</i> 16.	21
A.17. <i>Burndown chart</i> del <i>sprint</i> 17.	22
A.18. <i>Burndown chart</i> del <i>sprint</i> 18.	23
A.19. <i>Burndown chart</i> del <i>sprint</i> 19.	24
A.20. <i>Burndown chart</i> del <i>sprint</i> 20.	25
A.21. <i>Burndown chart</i> del <i>sprint</i> 21.	27
A.22. <i>Burndown chart</i> del <i>sprint</i> 22.	28
A.23. <i>Burndown chart</i> del <i>sprint</i> 23.	29
A.24. <i>Burndown chart</i> del <i>sprint</i> 24.	30
A.25. <i>Burndown chart</i> del <i>sprint</i> 25.	31
B.1. Diagrama general de casos de uso.	43

C.1. Diagrama Entidad/Relación.	67
C.2. Diagrama relacional.	68
C.3. Diagrama de la comunicación entre los principales componentes del sistema.	70
C.4. Diagrama de la gestión de peticiones del sistema.	71
C.5. Arquitectura cliente-servidor multicapa.	71
C.6. Diagrama de la arquitectura Modelo-Vista-Controlador (MVC) del sistema.	73
C.7. Diagrama de la arquitectura del modelo <i>RAG</i>	75
C.8. Prototipo de la página de inicio.	76
C.9. Prototipo de la página de inicio de sesión.	77
C.10. Prototipo de la página de registro.	77
C.11. Prototipo de la página principal.	78
C.12. Prototipo de la página del historial.	78
C.13. Prototipo de los detalles del historial.	79
C.14. Prototipo de la página de consulta.	79
C.15. Prototipo de la página de gestión de usuarios.	80
C.16. Prototipo de la página de añadir usuarios.	80
C.17. Prototipo de la página de edición del usuario.	81
C.18. Prototipo de la página de gestión de documentos.	81
C.19. Diseño de interfaz – Cabecera.	82
C.20. Diseño de interfaz – Historial de consultas.	83
C.21. Diseño de interfaz – Tabla de documentos cargados.	84
C.22. Diseño de interfaz – Interfaz de consulta al modelo.	85
C.23. Diagrama de navegabilidad por la aplicación para el administrador.	86
C.24. Diagrama de navegabilidad por la aplicación para el usuario normal.	87
D.1. Diagrama de flujo de la recuperación semántica tras una petición al sistema.	98

Índice de tablas

A.1. Tiempo estimado de los <i>story points</i>	2
A.2. Coste total de desarrollo (TCO).	33
A.3. Coste total del proyecto.	36
B.1. CU-1 Registro de usuarios.	44
B.2. CU-2 Iniciar sesión.	45
B.3. CU-3 Cerrar sesión.	46
B.4. CU-4 Recuperar contraseña.	47
B.5. CU-5 Realizar consulta al <i>RAG</i>	48
B.6. CU-6 Consultar historial de consultas.	49
B.7. CU-7 Borrar consultas del historial.	50
B.8. CU-8 Gestión de documentos.	51
B.9. CU-9 Cargar documentos de licitaciones.	52
B.10. CU-10 Importar licitaciones automáticamente.	53
B.11. CU-11 Convertir los documentos a <i>Markdown</i>	54
B.12. CU-12 Indexar documento.	55
B.13. CU-13 Listar documentos.	56
B.14. CU-14 Consultar estado del documento.	57
B.15. CU-15 Eliminar documento.	58
B.16. CU-16 Descargar documento.	59
B.17. CU-17 Gestión de usuarios.	60
B.18. CU-18 Crear usuario.	61
B.19. CU-19 Borrar usuario.	62
B.20. CU-20 Modificar rol del usuario.	63
B.21. CU-21 Editar usuario.	64

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este documento define cómo se va a gestionar y ejecutar el desarrollo de un producto o sistema informático. Establece el alcance, los objetivos, el cronograma, los recursos, el presupuesto, los riesgos y los criterios de calidad, así como la organización del equipo y los mecanismos de seguimiento y control. En esencia, convierte una idea de software en un itinerario gestionable, con responsabilidades claras y procedimientos para controlar el progreso y los cambios.

Su importancia radica en que reduce la incertidumbre y aumenta la probabilidad de éxito académico y técnico. Permite justificar decisiones, anticipar problemas, y demostrar competencias en gestión de proyectos, ingeniería de requisitos, calidad y comunicación técnica.

En este documento se va a incluir la planificación temporal de las fases y tareas del proyecto. Para cada una se debe establecer una fecha de inicio y se debe estimar la fecha final. Para realizar esto hay que tener en cuenta el peso de cada tarea, los requisitos previos y las dependencias.

También, se va a valorar la viabilidad del proyecto. En concreto, la viabilidad económica y la legal. En la viabilidad económica se estiman los costes y los beneficios del proyecto. Mientras que en la viabilidad legal analizan las leyes y licencias que afecten al desarrollo.

A.2. Planificación temporal

Para la planificación se utiliza una metodología Scrum, aplicando iteraciones (*sprints*). Al finalizar cada *sprint* se realiza una reunión para la revisión del incremento y la planificación del siguiente. En la planificación se seleccionan y valoran las tareas (*issues*) a realizar en el siguiente *sprint*. Estas tareas se han valorado usando los *story points* de ZenHub. A las tareas que se subdividen en otras se les ha asignado solo puntos extra respecto a sus subtareas, los *story points* no se han considerado acumulativos.

Para simplificar la asignación de los *story points* se les ha asignado una estimación temporal recogida en la tabla A.1:

<i>Story Points</i>	Tiempo estimado (horas)
1	0,5
2	1
3	2
5	4
8	6
13	9
21	15
40	más de 20

Tabla A.1: Tiempo estimado de los *story points*.

Sprint 0 (11/09/2025 – 18/09/2025)

Este *sprint* marcó el comienzo del proyecto. Los objetivos fueron realizar una primera toma de contacto con los *RAG* y los *LLM*, investigando sobre qué son, cómo funcionan y qué ventajas aportan los *RAG*. También, se planteó empezar a escribir los conceptos teóricos relevantes y crear el repositorio [GitHub](https://github.com/lbr1014/TFG_RAG.git)¹ para el proyecto. Como el *sprint* se diseñó antes de la creación del repositorio, no se creó el *milestone* asociado como se ha hecho en los *sprints* posteriores. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Crear el repositorio en GitHub.
- Investigar sobre los *RAG* y *LLM*.

¹https://github.com/lbr1014/TFG_RAG.git

- Leer el capítulo 1 del libro: *A simple guide to retrieval augmented generation* [6].
- Leer el capítulo 2 del libro: *A simple guide to retrieval augmented generation* [6]
- Escribir conceptos teóricos sobre lo investigado.

Para realizar estas tareas se estimaron 6 horas de trabajo pero en realidad fueron 9. Al finalizar el *sprint* se completaron todas las tareas.

Sprint 1 (18/09/2025 – 25/09/2025)

Los objetivos del **Sprint 1**² fueron investigar sobre los *LLM* y los *RAG* y escribir la información más relevante como conceptos teóricos en la memoria. Comenzar a escribir los objetivos del proyecto y las técnicas y herramientas definidas hasta el momento, como la metodología Scrum o el repositorio. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Escribir objetivos del proyecto y técnicas y herramientas.
- Escribir conceptos teóricos.
 - Investigar sobre los *RAG* y los *LLM*.
 - Leer el capítulo 3 del libro: *A simple guide to retrieval augmented generation* [6].
 - Leer el capítulo 4 del libro: *A simple guide to retrieval augmented generation* [6].

Para realizar estas tareas se estimaron 8,5 horas de trabajo pero en realidad fueron 10. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.1.

Sprint 2 (25/09/2025 – 02/10/2025)

Los objetivos del **Sprint 2**³ fueron seguir investigando sobre los *LLM* y *RAG* y explicar la información relevante en los conceptos teóricos. Investigar sobre el *Web Scraping* y explicar en los anexos el plan de proyecto, en concreto la planificación temporal y los *sprints*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

²https://github.com/lbr1014/TFG_RAG/milestone/1?closed=1

³https://github.com/lbr1014/TFG_RAG/milestone/2?closed=1



Figura A.1: *Burndown chart* del *sprint* 1.

- Añadir las correcciones a la memoria.
- Completar el apartado *pipeline* de generación del apartado conceptos teóricos.
- Investigar sobre *Web Scraping*.
 - Investigar sobre HTML.
 - Investigar sobre CSS.
- Explicar el plan de proyectos en los anexos.

Para realizar estas tareas se estimaron 10 horas de trabajo pero en realidad fueron 12. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.2.

***Sprint* 3 (2/10/2025 – 09/10/2025)**

Los objetivos del **Sprint 3**⁴ fueron realizar un primer prototipo de *Web Scraping*, tanto en *Selenium* como en *Playwright*. Incluir en la memoria las correcciones y la información relevante. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

⁴https://github.com/lbr1014/TFG_RAG/milestone/3?closed=1

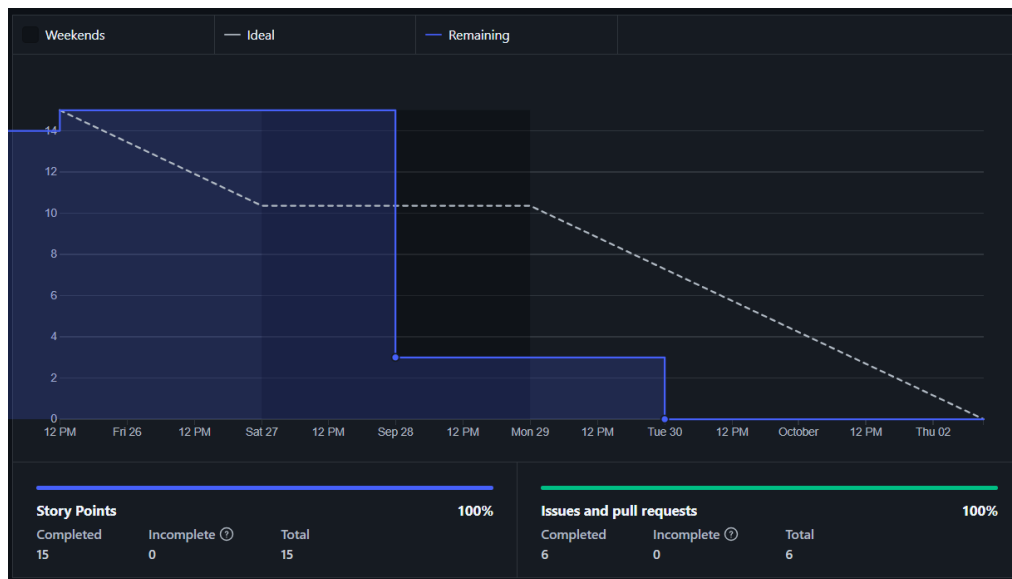


Figura A.2: *Burndown chart* del *sprint* 2.

- Añadir las correcciones a la memoria.
- Realización del comienzo de la aplicación de *web scraping*.
 - Investigar *web scraping* con Selenium.
 - Investigar *web scraping* con Playwright.
- Escribir conceptos teóricos en la memoria.
 - Leer el capítulo 5 del libro: *A simple guide to retrieval augmented generation* [6].

Para realizar estas tareas se estimaron 15 horas de trabajo pero en realidad fueron 18. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.3.

***Sprint* 4 (9/10/2025 – 16/10/2025)**

Los objetivos del **Sprint 4**⁵ fueron continuar realizando la aplicación de *Web Scraping*, tanto en *Selenium* como en *Playwright*, para sacar licitaciones de la página de **Contratación del Sector Público**⁶. Incluir en la memoria una

⁵https://github.com/lbr1014/TFG_RAG/milestone/4?closed=1

⁶<https://tinyurl.com/48ay5679>

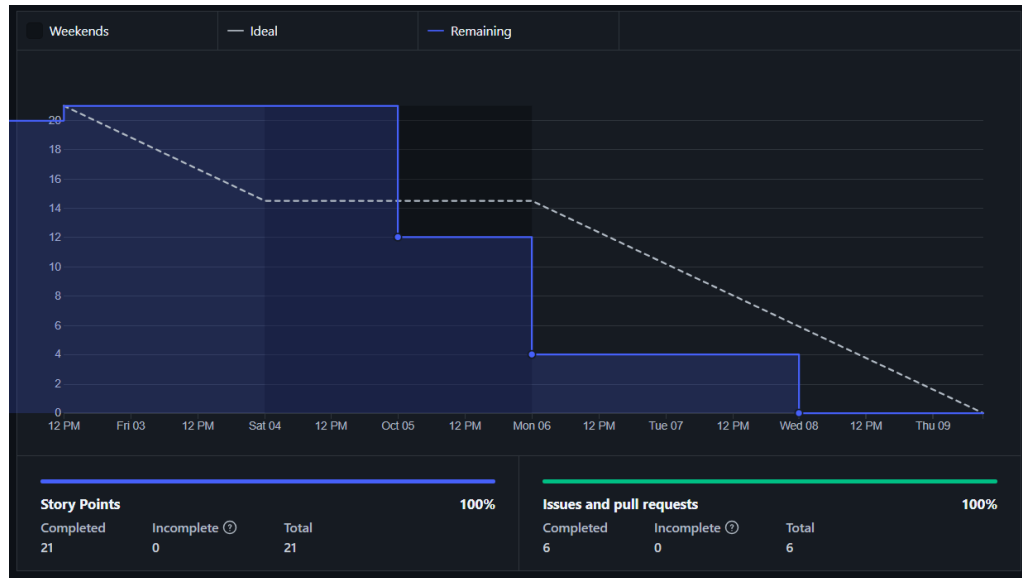


Figura A.3: *Burndown chart* del *sprint* 3.

comparación entre ambas aplicaciones de *Web Scraping*. Seguir investigando sobre los *RAG*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Continuar con el *script* de *web scraping* usando Playwright.
 - Continuar con el *script* de *web scraping* usando Selenium.
 - Estudiar sobre JSON.
- Escribir conceptos teóricos en la memoria.
 - Leer el capítulo 6 del libro: *A simple guide to retrieval augmented generation* [6].

Para realizar estas tareas se estimaron 14 horas de trabajo pero en realidad fueron 18. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.4.

***Sprint* 5 (16/10/2025 – 23/10/2025)**

Los objetivos del **Sprint 5**⁷ fueron investigar sobre las mejoras que aporta la *API* asíncrona de *Playwright* y como se implementa. También, se

⁷https://github.com/lbr1014/TFG_RAG/milestone/5?closed=1

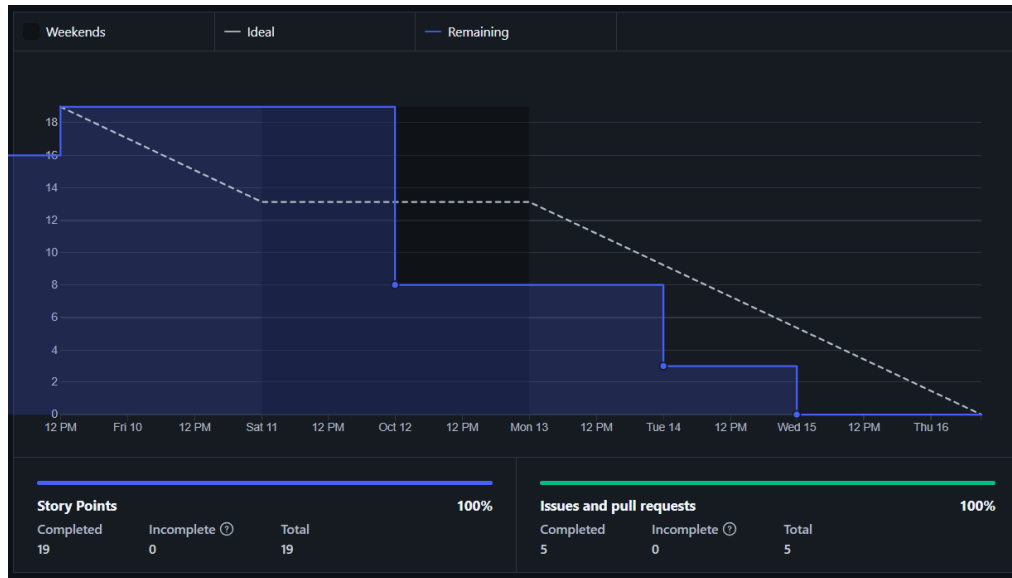


Figura A.4: *Burndown chart* del *sprint* 4.

planificado investigar sobre la gestión de dependencias y entornos en *Python*. Todo esto mientras se sigue investigando sobre los *RAG* y *LLM* y se escriben los aspectos más relevantes en la memoria. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Realizar código con *Playwright* asíncrono.
 - Investigar sobre la *API* asíncrona de *Playwright*.
- Investigar sobre las dependencias y los entornos en *Python*.
- Escribir conceptos teóricos en la memoria.
 - Leer más capítulo del libro: *A simple guide to retrieval augmented generation* [6].

Para realizar estas tareas se estimaron 15 horas de trabajo pero en realidad fueron 18, ya que se tuvo que reescribir parte del código realizado en los *sprints* anteriores para el *Web Scraping* debido a una actualización en la página de **Contratación del Sector Público**⁸. Este cambio afectó en mayor medida a la extracción de información de las licitaciones, ya que ha sido la página que más ha cambiado con dicha actualización. A pesar de este contratiempo al finalizar el *sprint* se completaron todas las tareas.

⁸<https://tinyurl.com/48ay5679>



Figura A.5: *Burndown chart* del *sprint* 5.

El *burndown* de este *sprint* se ilustra en la figura A.5.

***Sprint* 6 (23/10/2025 – 30/10/2025)**

Los objetivos del **Sprint 6**⁹ fueron descargar los documentos (en formato PDF) de las licitaciones de la página de **Contratación del Sector Público**¹⁰. Para descargar dichos PDF se va a usar de base el programa de *Web Scraping* del *sprint* anterior. Además, se han fijado como objetivos investigar sobre los modelos de inteligencia artificial de *Hugging Face* e instalar y probar el funcionamiento de *Ollama*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Investigar sobre *Ollama*.
- Investigar y probar modelos de inteligencia artificial de *Hugging Face*.
- Investigar sobre formas de implementar los *RAG*.
- Descargar los pdfs de las licitaciones de la la página de **Contratación del Sector Público**¹¹.
- Instalar las dependencias en *Poetry*.

⁹https://github.com/lbr1014/TFG_RAG/milestone/6?closed=1

¹⁰<https://tinyurl.com/48ay5679>

¹¹<https://tinyurl.com/48ay5679>



Figura A.6: *Burndown chart* del *sprint* 6.

Para realizar estas tareas se estimaron 12 horas de trabajo pero en realidad fueron 15, se ha usado un servidor de la UBU para descargar un zip con las licitaciones. Para acceder a dichas licitaciones se ha usado el archivo «resultados_playwright_asincrono» obtenido en el *sprint* anterior, en el cual se almacenan los enlaces a los pdfs de cada licitación. Al finalizar el *sprint* se completaron todas las tareas.

El *burndown* de este *sprint* se ilustra en la figura A.6.

***Sprint* 7 (30/10/2025 – 6/11/2025)**

El objetivo principal del **Sprint 7**¹² fue investigar e implementar distintos tipos de *tokenizer*. Esto se planteó como un primer paso para la implantación total del *RAG*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Investigar sobre los *tokenizers*.
- Implementar distintos tipos de *tokenizer*.
- Comparar las implementaciones de *tokenizers*.

¹²https://github.com/lbr1014/TFG_RAG/milestone/7?closed=1



Figura A.7: *Burndown chart* del *sprint* 7.

Para realizar estas tareas se estimaron 12 horas de trabajo pero en realidad fueron 15. En este *sprint* se han llevado a cabo todas las tareas planificadas, pero sería destacable que en los *scripts* de las diversas pruebas de *tokenizers* no solo se han incluido estos. Si no que también se han incluido funciones para probar su uso, como pueden ser *prompts* y funciones para dividir los pdfs de las licitaciones.

El *burndown* de este *sprint* se ilustra en la figura A.7.

***Sprint* 8 (6/11/2025 – 13/11/2025)**

El objetivo principal del **Sprint 8**¹³ fue implementar una base de datos para el *RAG* y juntarlo con uno de los *tokenizers* ya implementados del *sprint* anterior. Además, en este *sprint* se planteó como objetivo obtener los pliegos definitivos de de la página de .

Esto se debe a que los pliegos obtenidos anteriormente no eran correctos, se trataban de resúmenes incompletos que no servían para alimentar al *RAG*. Durante el *sprint* 7, se encontraron los pliegos correctos y se fijó como objetivo del siguiente *sprint* modificar los programa de *Web Scraping* para

¹³https://github.com/lbr1014/TFG_RAG/milestone/8?closed=1



Figura A.8: *Burndown chart* del *sprint* 8.

descargarlos. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Obtener pliegos correctos.
- Estudiar sobre las distintas bases de datos.
- Implementar la base de datos.
- Explicar la comparativa de las bases de datos.

Para realizar estas tareas se estimaron 12 horas de trabajo, pero en realidad fueron 16. En este *sprint* se han llevado a cabo todas las tareas planificadas. Se ha implementado la base de datos vectorial *Qdranty* y se ha juntado con el diseño de *tokenizer* del *sprint* anterior, aunque se ha cambiado el modelo para usar el del libro *LLM Engineer's Handbook*¹⁴. Además, se han realizado los *embeddings* para poder almacenarlos en la base de datos vectorial.

El *burndown* de este *sprint* se ilustra en la figura A.8.

¹⁴<https://www.amazon.com/LLM-Engineers-Handbook-engineering-production/dp/1836200072>

Sprint 9 (13/11/2025 – 20/11/2025)

El objetivo principal del **Sprint 9**¹⁵ fue realizar un *script* que, dada una consulta, devuelva el *chunk* más parecido. Junto con este *chunk* debe devolver el nombre y el título del PDF, así como una respuesta del *LLM* basándose en dicho *chunk*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Implementar funciones que devuelvan el nombre del PDF al que pertenece el *chunk*.
- Mejorar la partición de los *chunks* añadiendo solapamiento (*overlap*).
- Obtención del fragmento del *chunk*.
- Implementar código para que el *LLM* de una respuesta a partir de una pregunta del usuario.
- Añadir al código funciones para medir los tiempos.
- Usar *loggers* en el código.

Para realizar estas tareas se estimaron 18 horas y más o menos se ha cumplido dicha estimación. En este *sprint* se han llevado a cabo todas las tareas planificadas. Se ha mejorado el código añadiendo medidas de los tiempos de ejecución e implementando el uso de *loggers*. También, se ha añadido solapamiento en los *chunks* lo que mejora la eficacia de estos. Por otro lado, se ha implementado el objetivo principal, es decir, el *script* que permite que el usuario realice una pregunta y el *LLM* conteste basándose en el *chunk* más parecido a la pregunta. Además se indica el pdf del que ha sacado la información así como el *chunk* concreto.

El *burndown* de este *sprint* se ilustra en la figura A.9.

Sprint 10 (20/11/2025 – 27/11/2025)

El objetivo principal del **Sprint 10**¹⁶ fue intentar convertir los PDF a *Markdown*. Durante la reunión de planificación del *sprint* se decidió intentarlo para ver si el conversor a *Markdown* es capaz de detectar la estructura jerárquica de los documentos PDF, ya que, si detectará las secciones y subsecciones se podrían añadir como metadatos. Esto mejoraría significativamente la relevancia de los *chunks* al poder contextualizarlos mejor. Se han valorado diversas formas de realizar la conversión, por ejemplo, desde un *OCR* vinculado a un *LLM* o directamente empleando una biblioteca de *Python*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

¹⁵https://github.com/lbr1014/TFG_RAG/milestone/9?closed=1

¹⁶https://github.com/lbr1014/TFG_RAG/milestone/10?closed=1



Figura A.9: *Burndown chart* del *sprint* 9.

- Buscar formas de automatizar la conversión a *Markdown*.
- Realizar la conversión de un pdf a *Markdown*.
 - Automatizar la conversión a *Markdown*.

Para realizar estas tareas se estimaron 10 horas, pero debido a la gran heterogeneidad de formatos de los PDF fueron más, aproximadamente unas 15. A pesar de que se han obtenido resultados significativamente mejores que en las primeras aproximaciones, no se ha conseguido ningún *Markdown* lo suficientemente bueno. Esto se debe a que cada PDF de licitaciones tiene un formato distinto y no se ha conseguido ningún *script* que detecte correctamente todas las secciones.

El *burndown* de este *sprint* se ilustra en la figura A.10.

***Sprint* 11 (27/11/2025 – 4/12/2025)**

El objetivo principal del *Sprint* 11¹⁷ fue intentar implementar un *script* que basándose en los resultados obtenidos del anterior sea capaz de detectar correctamente las secciones de los PDF. Además, se propuso comenzar a investigar sobre el funcionamiento de *Flask*. Ya que esta aplicación se va a

¹⁷https://github.com/lbr1014/TFG_RAG/milestone/11?closed=1

Figura A.10: *Burndown chart* del *sprint* 10.

utilizar en el proyecto para realizar una *web* para realizar las consultas al *LLM*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Probar nuevos OCR para la conversión de los PDF a *Markdown*.
- Investigar sobre *Flask*.

Para realizar estas tareas se estimaron 8 horas. Al final se emplearon cerca de 12 horas debido al mismo problema del *sprint* anterior, que no se ha conseguido implementar un *script* capaz de detectar correctamente todas las secciones.

El *burndown* de este *sprint* se ilustra en la figura A.11.

***Sprint* 12 (4/12/2025 – 15/01/2026)**

El objetivo principal del **Sprint 12**¹⁸ fue comenzar a realizar la página *web* con *Flask* y comenzar a definir los casos de uso del proyecto. Este *sprint* fue más largo que el resto ya que coincidió con la semana de exámenes finales del primer cuatrimestre y las navidades. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

¹⁸https://github.com/lbr1014/TFG_RAG/milestone/12?closed=1



Figura A.11: *Burndown chart* del *sprint* 11.

- Definir los casos de uso.
 - Identificar los casos de uso del proyecto.
 - Identificar los requisitos del proyecto.
- Comenzar la página *web*.
- Hacer la gestión básica de usuarios.
- Implementar la gestión de usuarios para los administradores.
- Hacer los *test* para la gestión de usuarios de la *web*.

Para realizar estas tareas se estimaron 28 horas de trabajo. Al final se emplearon más de 35 horas, debido principalmente a que la tarea «Comenzar la página *web*» llevó más tiempo del planificado. Esta tarea consistió en realizar la estructura básica de la *web*, la cual finalmente se dividió en *blueprints* para que tuviese mejor escalabilidad. Comprender el funcionamiento de los *blueprints* y saber implementarlos aumentó significativamente el tiempo de desarrollo de la estructura de la *web*.

El *burndown* de este *sprint* se ilustra en la figura A.12.

Figura A.12: *Burndown chart* del *sprint* 12.

***Sprint* 13 (15/01/2026 – 22/01/2026)**

El objetivo principal del **Sprint 13**¹⁹ fue completar las funcionalidades de la *web* permitiendo que los administradores puedan cargar documentos, de forma manual o automática (mediante *web scraping*). El sistema gestionará estos documentos para extraer de ellos los *embeddings* necesarios para el *RAG*. Además, se marcó como objetivo incluir la interfaz para realizar las consultas al *LLM*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Añadir casos de uso.
- Incluir una clase consulta.
- Añadir la funcionalidad carga de documentos.
- Mostrar historial de consultas.
- Implementar las consultas en la *web Flask*.
- Juntar el *web scraping* con la carga de documentos de la *web*.
- Realizar la interfaz de las consultas.
- Poner la clave secreta como variable de entorno.

¹⁹https://github.com/lbr1014/TFG_RAG/milestone/13?closed=1



Figura A.13: *Burndown chart* del *sprint* 13.

Para realizar estas tareas se estimaron 28,5 horas de trabajo. Al final del *sprint* se cumplieron todas las tareas más o menos en el plazo esperado.

El *burndown* de este *sprint* se ilustra en la figura A.13.

***Sprint* 14 (22/01/2026 – 29/01/2026)**

El objetivo principal del **Sprint 14**²⁰ fue convertir los procesos de *web scraping* y de actualización de la base de datos vectorial a modo asíncrono. En la reunión se decidió priorizar este cambio, ya que la implementación anterior, en modo síncrono, tardaba mucho tiempo y bloqueaba la web para el usuario. Por otro lado, se decidió continuar desarrollando la documentación, concretamente los anexos. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Cambios en las clases de la aplicación.
 - Cambios en la clase documentos.
- Escribir en los anexos el diseño de datos.
 - Realizar el diagrama relacional.

²⁰https://github.com/lbr1014/TFG_RAG/milestone/14?closed=1

- Realizar el diagrama Entidad/Relación.
- Aplicar correcciones a los anexos.
- Realizar el diagrama de casos de uso.
- Diseño de interfaces.
- Cambio de funcionalidades a modo asíncrono.
- Incluir mensajes de *feedback* para el usuario.
- Incluir las clases **Chunk** y **Embedding**.
- Aplicar cambios en el código para integrar correctamente las nuevas clases.
- Crear tabla **ConsultaChunk**.
- Revisar y corregir código usando *SonarCloud*.
- Hacer que el *LLM* use más de un *chunk* para responder.

Para realizar estas tareas se estimaron 50 horas de trabajo. Al final del *sprint* se cumplieron todas las tareas. El tiempo final dedicado fue más o menos el estimado. A pesar de que algunas tareas (como las de corregir el código usando *SonarCloud* y las de cambiar las funcionalidades a modo asíncrono) se retrasaron, hubo otras como las de realizar los diagramas que se adelantaron.

El *burndown* de este *sprint* se ilustra en la figura A.14.

Sprint 15 (29/01/2026 – 05/02/2026)

El objetivo principal del **Sprint 15**²¹ fue levantar mi aplicación *Flask* usando un contenedor *docker*. En la reunión se decidió implementar un *docker* ya que mejora la potabilidad, la ligereza y el aislamiento de la aplicación. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Implementar un *docker* para la aplicación *Flask*.
 - Estudiar como desplegar la aplicación *web*.
 - Arreglar la indexación de documentos para usar el *docker*.
 - Arreglo de *Web Scraping* para evitar fallos por perdida de conexión.
- Empezar a escribir la viabilidad económica.
- Completar los *test* unitarios realizados previamente.
- Realiza *test* unitarios para las nuevas clases.

²¹https://github.com/lbr1014/TFG_RAG/milestone/15?closed=1

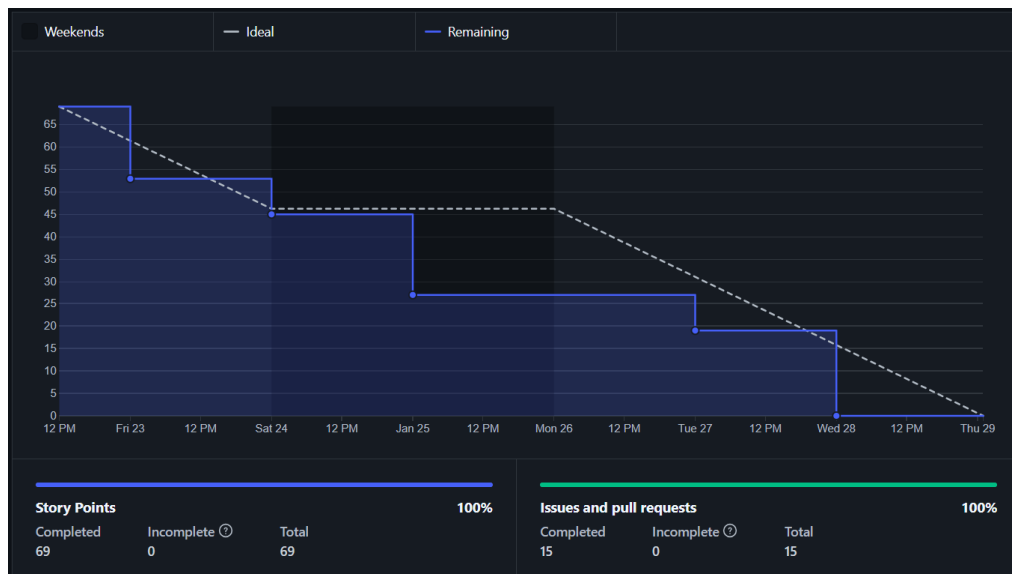


Figura A.14: *Burndown chart* del *sprint* 14.

Para realizar el *sprint* se estimaron 34 horas de trabajo. Al final de este *sprint* se invirtieron unas 40 horas, pero, no se completaron todas las tareas, quedando incompletas las tareas «Completar los *test* unitarios realizados previamente» y «Realiza *test* unitarios para las nuevas clases». Esto se debió a que la implementación de *docker* llevo más tiempo del esperado por fallos en la conexión con el módulo de *qdrant*. Las tareas incompletas se van a llevar a cabo en el siguiente *sprint*.

El *burndown* de este *sprint* se ilustra en la figura A.15.

***Sprint* 16 (05/02/2026 – 12/02/2026)**

El objetivo principal del **Sprint 16**²² fue incluir el uso de *Nginx* para levantar la aplicación *web* en el servidor integrándolo con la implementación de *Gunicorn* del *sprint* anterior. Además, se fijó como objetivo explicar en la memoria algunos trabajos relacionados. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Incluir trabajos relacionados en la memoria.
- Incluir imágenes en la memoria.

²²https://github.com/lbr1014/TFG_RAG/milestone/16?closed=1

Figura A.15: *Burndown chart* del *sprint* 15.

- Correcciones documentación.
- Completar algunos de los *tests* unitarios realizados previamente.
- Realizar *tests* unitarios para las nuevas clases.
- Mejoras en el *docker*.
- Incluir *Nginx* en la aplicación *Flask*.

Para realizar el *sprint* se estimaron 34 horas de trabajo. Al finalizar el *sprint* se realizaron todas las tareas, y de manera general respetando la planificación temporal. En el caso de la tarea «Incluir trabajos relacionados en la memoria» se estimó que se iba a realizar en 4 horas pero finalmente fueron unas 6 horas, ya que, se realizó una comparativa entre ellos y con el proyecto que se esta llevando a cabo en este TFG.

El *burndown* de este *sprint* se ilustra en la figura A.16.

***Sprint* 17 (12/02/2026 – 19/02/2026)**

El objetivo principal del *Sprint* 17²³ fue incluir que la aplicación pueda mandar correos a los usuarios, concretamente para avisar cuando acaban de ejecutarse los procesos largos (como el *web scraping* o la actualización de la

²³https://github.com/lbr1014/TFG_RAG/milestone/17?closed=1

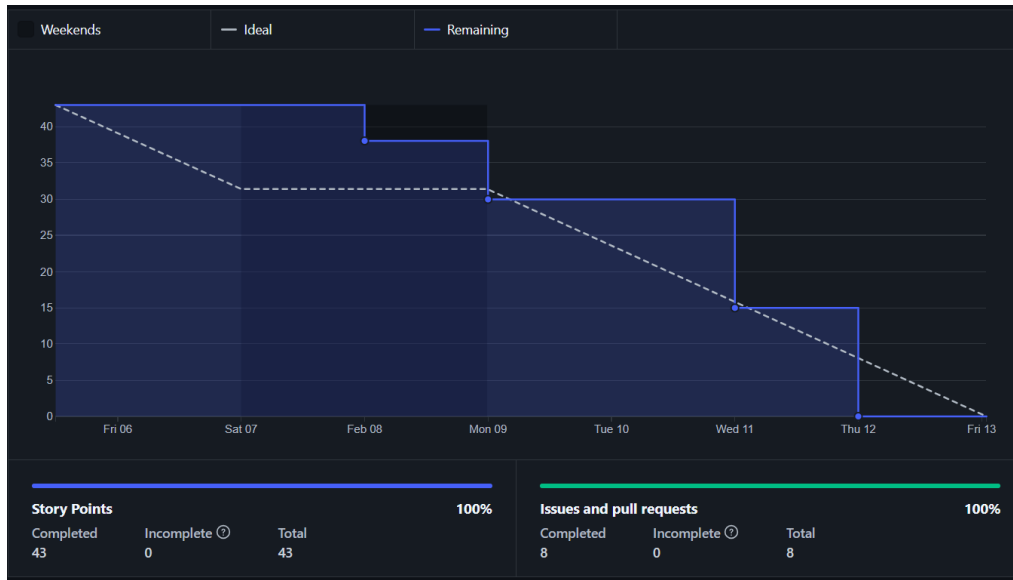


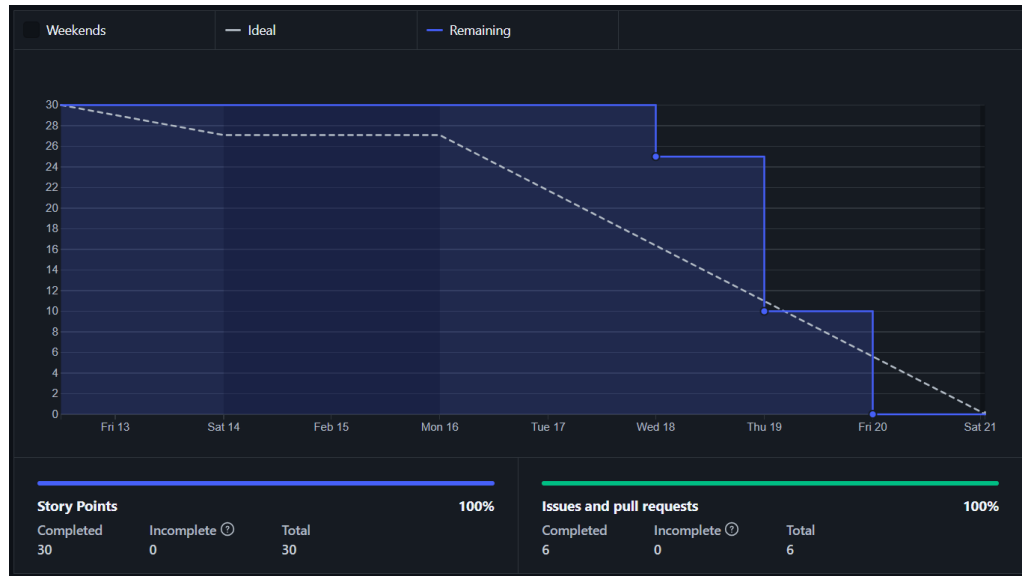
Figura A.16: *Burndown chart* del *sprint* 16.

base de datos vectorial) y para la recuperación de la contraseña cuando se le ha olvidado. También, se siguieron implementando los *tests* unitarios de la aplicación para cubrir el *coverage*. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Correo cuando finalicen las tareas largas.
 - Incluir recuperación de contraseña.
 - Investigar como se pueden mandar correos automáticos desde *Flask*.
- Completar *tests* y generar el informe del *coverage*.
- Incluir conversión a *markdown* en la aplicación.
- Botón cancelar consulta.

Para realizar el *sprint* se estimaron 24 horas de trabajo. Al finalizar el *sprint* se realizaron todas las tareas invirtiendo unas 30 horas. Esto se debió a que la realización de los *tests* llevó más tiempo del planificado. Sobre todo, aquellos los de las clases concurrentes y aquellos para los que se necesitaba entorno con bases de datos.

El *burndown* de este *sprint* se ilustra en la figura A.17.

Figura A.17: *Burndown chart* del *sprint* 17.

***Sprint* 18 (19/02/2026 – 26/02/2026)**

El objetivo principal del **Sprint 18**²⁴ fue documentar el repositorio de GitHub, para ello se crearon archivos **README** en los que se indica el contenido de cada directorio, así como la forma en la que se ejecutan. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Documentar la carpeta prototipos.
 - Documentar en GitHub la aplicación Flask con *Docker*.
 - Documentar los archivos de *Web Scraping*.
 - Documentar la carpeta *tokenizer*.
 - Documentar la carpeta *Markdown*.
 - Documentar el directorio *Flask*
- Explicar las herramientas para el *Docker* en la memoria.

Para realizar el *sprint* se estimaron 21 horas de trabajo. Al finalizar el *sprint* se realizaron todas las tareas invirtiendo unas 26 horas. Debido a que las explicaciones de los prototipos llevaron más tiempo del estimado, sobre todo, la explicación de los archivos del directorio prototipos.

El *burndown* de este *sprint* se ilustra en la figura A.18.

²⁴https://github.com/lbr1014/TFG_RAG/milestone/18?closed=1



Figura A.18: *Burndown chart* del *sprint* 18.

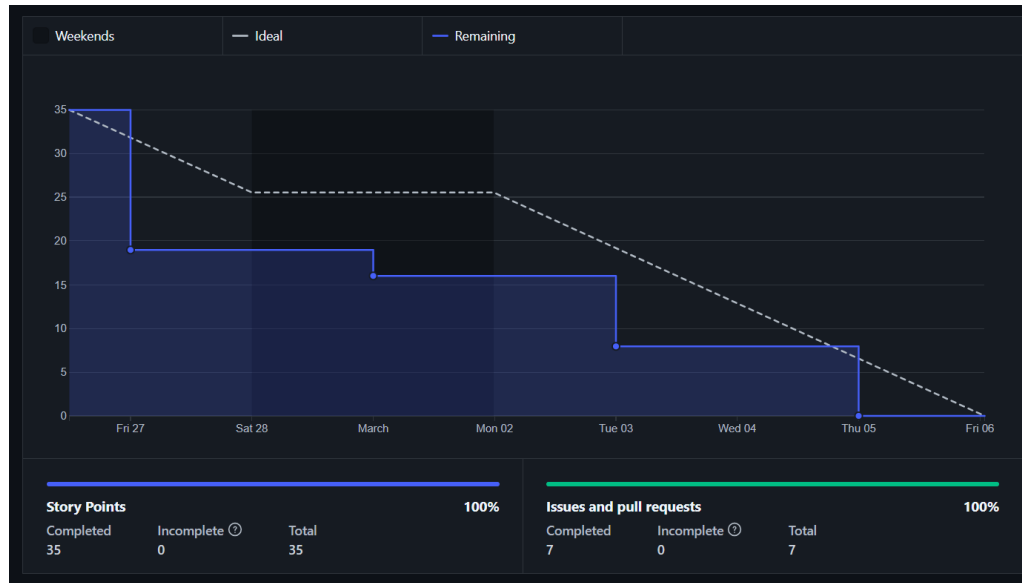
***Sprint* 19 (26/02/2026 – 5/03/2026)**

Los objetivos que se fijaron para el **Sprint 19²⁵** se dividieron en dos. Por un lado, se propuso configurar la *web* para que al acceder desde *Internet* tuviera nombre de dominio y siguiera el protocolo **HTTPS**. Además, se decidió comenzar a montar la aplicación definitiva (para instalar en local) en el repositorio en una nueva carpeta llamada *fileapp* y utilizando una rama de *git* para su edición con el mismo nombre. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Configurar DNS para mi aplicación.
 - Expedir un certificado **SSL/TLS** para mi *web*.
 - Conectar el certificado **SSL/TLS** con *Nginx*.
- Explicar la arquitectura de mi aplicación en los anexos.
- Reorganizar el prototipo de la aplicación *Flask*.
- Realizar diagramas explicativo de como se despliega la aplicación.
- Escribir introducción en la memoria.

Para realizar el *sprint* se estimaron 26 horas de trabajo. Al finalizar el *sprint* se realizaron todas las tareas cumpliendo más o menso el plazo

²⁵https://github.com/lbr1014/TFG_RAG/milestone/19?closed=1

Figura A.19: *Burndown chart* del *sprint* 19.

indicado. Aunque, individualmente no todas las tareas se completaron en el tiempo estimado. Por ejemplo, la tarea «Configurar DNS para mi aplicación» llevó más tiempo del esperado. Esto se debió a que la aplicación está levantada en un servidor con dirección privada y las direcciones privadas no son accesibles desde *Internet*. Peor el omputo general si que dio las 26 horas debido a que otras tareas como la «Reorganizar el prototipo de la aplicación *Flask*» llevaron menos tiempo del previsto.

El *burndown* de este *sprint* se ilustra en la figura A.19.

***Sprint* 20 (05/03/2026 – 12/03/2026)**

Como objetivos principales del **Sprint 20**²⁶ se fijaron realizar cambios en la interfaz gráfica de la *web* y en el *pipeline* de indexación del *RAG*. En la reunión se fijo que todos los elementos de la interfaz debían ser o utilizar elementos de *Bootstrap*, para hacer que la interfaz sea completamente *responsive*. Respecto al *pipeline* de indexación del *RAG* se decidió introducir la conversión a **Markdown**. Para en un futuro *sprint* hacer que los *chunks* y *embeddings* se obtengan desde los documentos **Markdown** y no los PDF. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

²⁶https://github.com/lbr1014/TFG_RAG/milestone/20?closed=1



Figura A.20: *Burndown chart* del *sprint* 20.

- Hacer la página de inicio de la aplicación.
- Adaptar la plantilla base para la *web*.
- Hacer que los componentes de la *web* sean *responsive*.
- Incluir acciones en la tabla documentos.
- Incluir la posibilidad de cancelar las tareas largas.
- Incluir la conversión a *Markdown*.

Para realizar el *sprint* se estimaron 20 horas de trabajo. Al finalizar el *sprint* se realizaron todas pero se invirtió más tiempo del planificado, unas 24 horas. este aumento se debió, principalmente, a que a la vez que se hacían *responsive* todos los elementos, se decidió cambiar el estilo de la interfaz *web* para que fuese similar al que se diseñó para la página de inicio.

El *burndown* de este *sprint* se ilustra en la figura A.20.

***Sprint* 21 (12/03/2026 – 19/03/2026)**

Como objetivo principal del **Sprint 21**²⁷ se fijó incluir en el *pipeline* de indexación documental la conversión a *Markdown* del *sprint* anterior. Además,

²⁷https://github.com/lbr1014/TFG_RAG/milestone/21?closed=1

de incluir correcciones en la memoria y seguir completando sus apartados. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Incluir la posibilidad del modo claro.
- Incluir en la memoria los aspectos relevantes del desarrollo del proyecto.
- Correcciones documentación.
- Incluir la conversión a **Markdown** en el *pipeline* de indexación documental.

Para realizar el *sprint* se estimaron 18 horas de trabajo. Sin embargo, se invirtieron más de 23 horas de trabajo y no se completaron todas las tareas. La tarea «Incluir la conversión a **Markdown** en el *pipeline* de indexación documental» quedó incompleta debido a varios fallos. El problema que no ha dado tiempo ha resolver y que se deja para el *sprint* siguiente es que al ser un proceso tan largo cualquier fallo aborta la conversión haciéndolo especialmente sensible a fallos de conexión y *timeouts*. Por otra parte la tarea «Incluir la posibilidad del modo claro» también supuso un retraso frente a la planificación inicial. Esto se debió a que para permitir el cambio entre modo claro y oscuro ha habido que incluir variables para todos los estilos y seleccionar los colores para el nuevo modo.

El *burndown* de este *sprint* se ilustra en la figura [A.21](#).

Sprint 22 (19/03/2026 – 26/03/2026)

Como objetivo principal del **Sprint 22**²⁸ se fijó terminar de arreglar la conversión a **Markdown**. Además, durante la reunión se analizó el funcionamiento del programa y se determinó que siempre que fuera posible la consulta se tenía que realizar utilizando la GPU para que el sistema conteste más rápido en lugar de usar la CPU. Respecto a la interfaz de usuario se decidió incluir la posibilidad de traducir la página y sus elementos al inglés. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Hacer que el modelo use la GPU.
- Incluir internacionalización.
- Incluir la conversión a **Markdown** en el *pipeline* de indexación documental.
- Corregir los fallos del **Markdown**.

²⁸https://github.com/lbr1014/TFG_RAG/milestone/22?closed=1

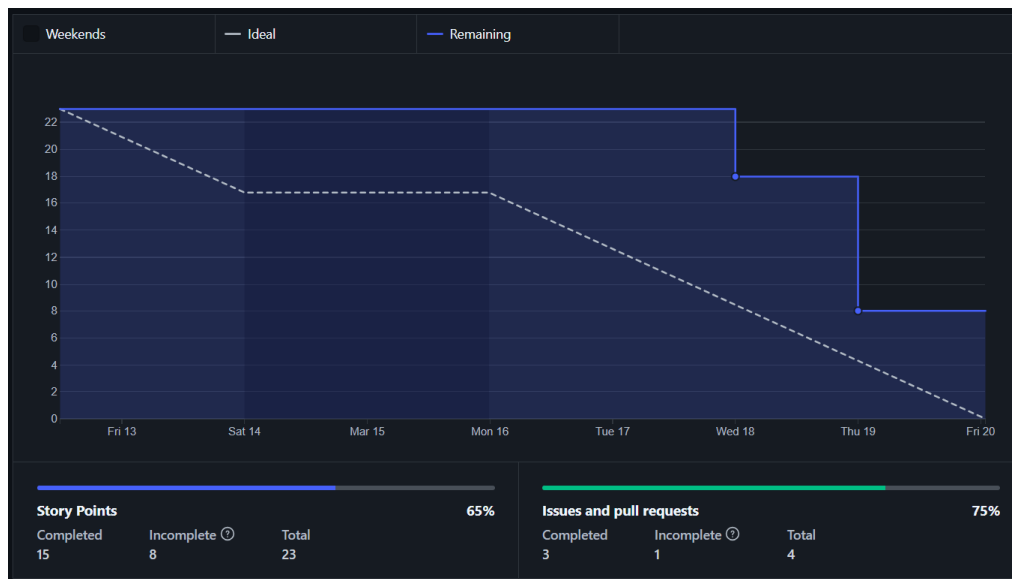


Figura A.21: *Burndown chart* del *sprint* 21.

- Incluir correcciones en el código de los prototipos.

Para realizar el *sprint* se estimaron 26 horas de trabajo. Al final se invirtieron más de 30 horas ya que corregir los fallos de la conversión a **Markdown** costó más de lo previsto. A pesar de que se ha mejorado el funcionamiento solventando problemas por *timeout* o páginas no convertidas correctamente, aún presenta algún problema que se solventará en futuros *sprints*. Por ejemplo, no captura bien los títulos y a veces se para en las conversiones largas.

El *burndown* de este *sprint* se ilustra en la figura A.22.

***Sprint* 23 (26/03/2026 – 09/04/2026)**

Como objetivo principal del **Sprint 23**²⁹ se fijó hacer que la conversión a **Markdown** se haga con la GPU en lugar de hacerse con la CPU como hasta ahora. Se ha considerado especialmente importante para reducir el tiempo dedicado a la conversión a **Markdown** de los PDF usados como contexto. Además, se decidió implementar gráficas de uso de la aplicación para los usuarios. Estas gráficas deben mostrar datos distintos a los usuarios normales,

²⁹https://github.com/lbr1014/TFG_RAG/milestone/23?closed=1

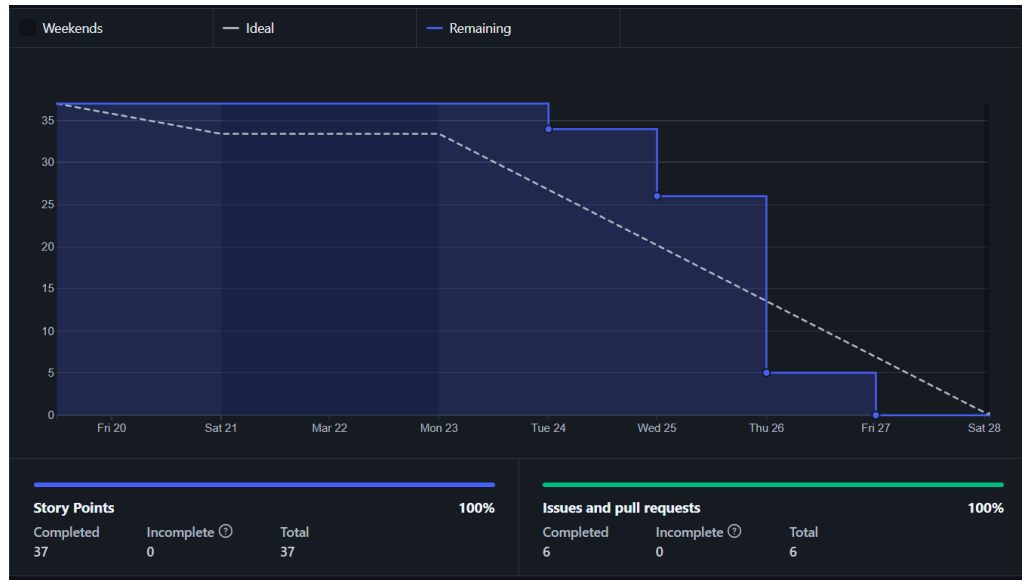


Figura A.22: *Burndown chart* del sprint 22.

que solo deben ver su uso de la aplicación, y a los administradores, que deben poder ver el uso de la aplicación de cualquier usuario. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Hacer que la conversión a **Markdown** use la GPU.
- Incluir gráficas en la *web*.
 - Diseñar gráficas relevantes para el cliente.
 - Estudiar el uso de D3 para generar gráficas
- Incluir manejo de errores.
 - Hacer *templates* para el manejo de errores.
- Cambiar los metadatos guardados para el *chunk*.
- Usar los metadatos de los *chunks* mejorar el RAG.
- Arreglar los *test*.

Para realizar el *sprint* se estimaron 40 horas de trabajo. Al finalizar el *sprint* se completaron todas las tareas más o menos en el plazo estimado. Como aspecto destacable cabe mencionar que a pesar de ejecutar la conversión a **Markdown** con la GPU sigue siendo muy lenta. Esto conlleva que todavía se paran las conversiones de los PDF más largos. Este problema se solucionará en *sprints* posteriores.

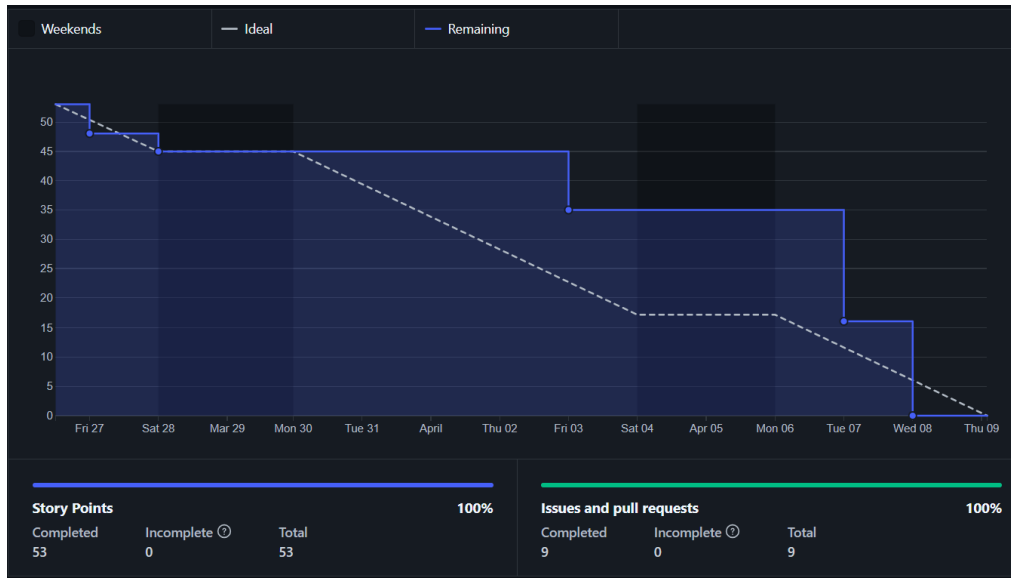


Figura A.23: *Burndown chart* del *sprint* 23.

El *burndown* de este *sprint* se ilustra en la figura A.23.

***Sprint* 24 (09/04/2026 – 16/04/2026)**

Como objetivo principal del **Sprint 24**³⁰ se fijó completar la documentación del código de la aplicación para que sea más entendible lo que hace cada *script* y cada método. Además, se decidió resolver las sugerencias de SonarCloud para que el código sea de mayor calidad, garantizando su calidad. Respecto al propio RAG se decidió permitir que use distintos modelos LLM para responder según cual seleccione el usuario. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Resolver las *issues* de SonarCloud.
- Documentar el código de la aplicación.
- Incluir modelos adicionales en la aplicación.

Para realizar el *sprint* se estimaron 18 horas de trabajo. Al finalizar el *sprint* si que se finalizaron todas las tareas, pero no se cumplió la estimación temporal. Tanto la documentación como la resolución de las *issues* de SonarCloud llevaron mucho más tiempo del estimado. Finalmente se invirtieron

³⁰https://github.com/lbr1014/TFG_RAG/milestone/24?closed=1



Figura A.24: *Burndown chart* del *sprint* 24.

unas 25 horas de trabajo. En la documentación se invirtieron 2 horas más de lo previsto y en resolver las *issues* 5 horas más.

El *burndown* de este *sprint* se ilustra en la figura A.24.

***Sprint* 25 (16/04/2026 – 30/04/2026)**

Como objetivo principal del **Sprint 25**³¹ se decidió incluir un formulario para las interacciones con el modelo que permita seleccionar pliegos concretos y hacer preguntas más específicas, así como resúmenes. Esto va a permitir optimizar las respuestas del modelo a dichas preguntas decidiendo mejor el número de *chunks* recuperados. Además se decidió que incluir gráficas de comparativa de modelos aportaría una ayuda al usuario para elegir el modelo que le responda a las preguntas. También, se decidió mejorar las gráficas de uso de los usuarios haciéndolas interactivas. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Incluir un formulario para las interacciones con el modelo.
 - Diseñar el *template* para el formulario de interacción con el modelo.

³¹https://github.com/lbr1014/TFG_RAG/milestone/25?closed=1

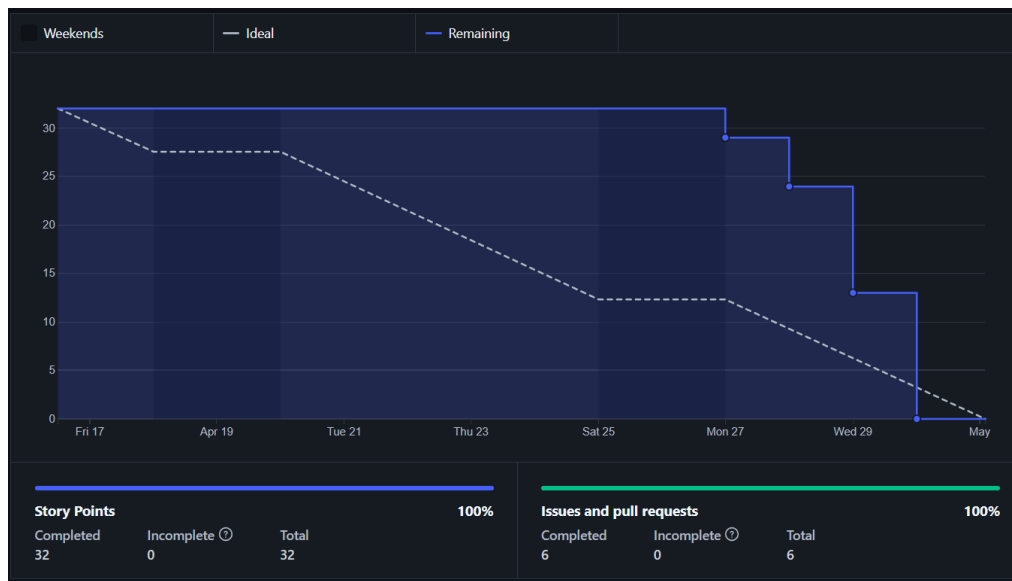


Figura A.25: *Burndown chart* del *sprint* 25.

- Realizar *prompts* específicos para las preguntas del formulario.
- Realizar gráficas para comparar los modelos.
 - Diseñar gráficas.
- Nuevo campo en consultas.

Para realizar el *sprint* se estimaron 24 horas de trabajo. Al finalizar el *sprint* se finalizaron todas las tareas, sin embargo, la parte de las consultas por defecto con el sistema implicó más cambios de los pensados ampliando el tiempo que costó realizarlo. Además, del diseño de *prompts* específicos se decidió cambiar la forma de recuperar los *chunks*. Dependiendo de la pregunta se pasa un número máximo de *chunks* recuperados distintos. Para todas las consultas se ha hecho que se recuperen un mínimo de 5 *chunks* hasta el máximo indicado para ese tipo de pregunta, pero solo recuperará aquellos con una similitud mayor del 0,5 para evitar introducir ruido al modelo.

El *burndown* de este *sprint* se ilustra en la figura A.25.

Sprint 26 (30/04/2026 – 07/05/2026)

Como objetivo principal del **Sprint 26**³² se decidió explicar algunos de los apartados que faltan de los anexos y la memoria. Concretamente se va a explicar el manual del programador, y se va a comenzar con el manual del usuario. También, se va a seguir redactando la viabilidad y los aspectos importantes durante el desarrollo. Para llevar a cabo estos objetivos se plantearon las siguientes tareas:

- Redactar la documentación técnica del programador.
 - Explicar las pruebas del sistema.
 - Explicar la forma de ejecutar el sistema en los anexos.
 - Redactar el manual del programador.
- Completar la viabilidad del proyecto.
- Documentación de usuario.
 - Redactar los requisitos de usuario.
 - Empezar a explicar el manual de usuarios.
- Explicar aspectos importantes del desarrollo del proyecto.

Para realizar el *sprint* se estimaron 32 horas de trabajo. Tras finalizar el *sprint* se explicarán todos los apartados en el tiempo estimado.

Sprint 27 (07/05/2026 – 14/05/2026)***Sprint 28 (14/05/2026 – 21/05/2026)******Sprint 29 (21/05/2026 – 28/05/2026)*****A.3. Estudio de viabilidad**

El estudio de viabilidad es un análisis detallado que se realiza antes de iniciar un proyecto. Sirve para determinar la rentabilidad de ese proyecto, es decir, si se puede hacer realidad. Para ello se analiza si es económicamente rentable (ver sección **A.3.1**) y legalmente viable (ver sección **A.3.2**).

³²https://github.com/lbr1014/TFG_RAG/milestone/26?closed=1

Viabilidad económica

Para analizar la viabilidad económica se ha seguido el coste total de propiedad (TCO) [7]. El TCO consiste en identificar y estimar todos los costes asociados al ciclo de vida de un producto tecnológico, no solo los costes directos de su adquisición, sino también los derivados de su operación, mantenimiento y uso a lo largo del tiempo. De esta forma, se obtiene una visión global y realista del impacto económico del sistema. Los costes considerados vienen recogidos en la tabla A.2.

Componentes	Componentes del costo
Adquisición <i>hardware</i>	Precio de compra del equipo <i>hardware</i> . Se debe tener en cuenta los ordenadores, terminales, almacenamiento e impresoras.
Adquisición <i>software</i>	Compra de licencias y <i>software</i> para cada usuario.
Instalación	Coste para instalar los ordenadores y el <i>software</i> .
Capacitación	Coste de entrenamiento de los especialistas y de los usuarios finales.
Soporte	Coste para dar soporte técnico continuo.
Mantenimiento	Coste para mantener y actualizar el <i>hardware</i> y <i>software</i> .
Infraestructura	Coste por adquirir, mantener y dar soporte a la infraestructura relacionada. Incluye las redes y el equipo especializado.
Tiempo inactivo	Coste de pérdidas de productividad debido a fallos de <i>hardware</i> o <i>software</i> . Estos fallos provocan que el sistema no este disponible.
Espacio y energía	Coste de bienes y servicios públicos para alojar y proveer energía para la tecnología.

Tabla A.2: Coste total de desarrollo (TCO).

Para llevar a cabo el estudio de la viabilidad económica se va a considerar que el trabajo llevado a cabo por la alumna lo ha realizado un desarrollador contratado durante los 9 meses que ha durado el proyecto. Si se supone un escenario empresarial real en España y se toma como referencia el **Salario Mínimo Interprofesional (SMI) de 2024**³³, el salario mínimo anual es de 15 876 €, es decir, 1 134 € al mes (se divide en 14 pagas). Además,

³³<https://www.boe.es/buscar/doc.php?id=B0E-A-2024-2251>

a este salario hay que añadirle los gastos por la cotización a la **Seguridad Social**³⁴. Normalmente la cotización es del 28,30 %, de los cuales 23,60 % se corresponden con gastos de la empresa. Por lo cual el gasto por la contratación del desarrollado sería:

$$1\,134 \frac{\text{€}}{\text{meses}} \cdot 1,236 \cdot 9 \text{ meses} = 12\,614,62 \text{ €}$$

Respecto a la adquisición de *hardware* no ha sido necesaria la adquisición de *hardware* específico adicional. El sistema *RAG* se ha implementado de forma que se pueda ejecutar en entornos estándares, sin requerir servidores dedicados ni equipamiento especializado. El desarrollo y las pruebas se han realizado utilizando mi equipo personal y servidores ya disponibles en la universidad. El equipo utilizado ha sido un **Victus by HP Gaming Laptop 16-r0xxx**³⁵. El coste de adquisición del portátil fueron unos 1 200 €. Considerando un periodo de amortización de 5 años, el coste amortizado correspondiente al proyecto con una duración de 9 meses (del 11 de septiembre al 11 junio) sería:

$$\left(\frac{1\,200 \text{ €}}{5 \text{ años} \cdot 12 \text{ meses}} \right) \cdot 9 \text{ meses} = 180 \text{ €}$$

Además, se ha usado el servidor **beta**³⁶ de la universidad. Aunque el servidor no se ha considerado un gasto de *hardware* ya que se ha considerado que se amortizaba en 5 años y ya tiene 9.

Respecto a la adquisición de *software* la mayor parte del *software* utilizado en el proyecto es de código abierto y de uso gratuito, como *Python*, *Flask*, *Qdrant*, *PostgreSQL*, *Docker* o las librerías de *Hugging Face*. Estas herramientas no implican un coste directo de licencia. Por lo cual el único gasto que debe considerarse en este apartado es la adquisición de la licencia de **Windows**³⁷ con un coste de unos 145 €. Se considera una amortización de 5 años, por lo cual para los 9 meses del proyecto:

$$\left(\frac{145 \text{ €}}{5 \text{ años} \cdot 12 \text{ meses}} \right) \cdot 9 \text{ meses} = 21,75 \text{ €}$$

Por otro lado en el proyecto no ha habido costes indirectos asociados a servicios de terceros como pueden ser las API o modelos en la nube, ya que se han elegido alternativas gratuitas.

³⁴<https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/36537>

³⁵https://support.hp.com/es-es/document/ish_7878690-7886844-16

³⁶<https://www.gigabyte.com/es/Enterprise/Server-Motherboard/MW51-HP0-rev-1x>

³⁷<https://www.microsoft.com/es-es/d/windows-11-home/dg7gmgf0krt0/000P>

Los costes de instalación incluyen el tiempo invertido en la configuración del entorno de desarrollo, instalación de dependencias, despliegue del sistema con *Docker* y configuración de las bases de datos. Este tiempo debe considerarse como un coste laboral. Se estima que en estas tareas se ha invertido un 20 % del tiempo total del proyecto. Por lo cual teniendo en cuenta que el coste laboral del desarrollador ha sido de 12 614,62 € el de los costes de instalación se corresponde:

$$12\,614,62\,€ \cdot 0,20 = 2\,522,92\,€$$

Respecto a los costes de capacitación, se ha requerido una fase importante de aprendizaje en tecnologías como *RAG*, bases de datos vectoriales, modelos *LLM*, *web scraping* y despliegue *web*. Este tiempo de formación constituye un coste indirecto de capital humano. Se estima que aproximadamente un 25 % del tiempo total del proyecto ha sido dedicado a tareas de aprendizaje. Por lo cual, el coste de capacitación ha sido:

$$12\,614,62\,€ \cdot 0,25 = 3\,153,66\,€$$

El soporte durante la ejecución del proyecto ha sido llevado a cabo por los tutores. Se han llevado a cabo reuniones (*sprints*) para la revisión de decisiones y la resolución de problemas. Han sido necesarias unas 28 reuniones a lo largo del proyecto de aproximadamente una hora cada una. Además, al tiempo invertido para el soporte técnico, debe sumarse el tiempo para llevar a cabo las revisiones de la memoria, los anexos y del código. En total se estima que los tutores han invertido unas 100 horas en el proyecto cada uno. Se va a suponer que su sueldo es de 20 € la hora al mes sería:

$$20 \frac{€}{\text{hora}} \cdot 7 \frac{\text{horas}}{\text{días}} \cdot 5 \frac{\text{días}}{\text{semana}} \cdot 4 \frac{\text{semanas}}{\text{mes}} = 2\,800 \frac{€}{\text{mes}}$$

Por lo cual el gasto en soporte sería:

$$20 \frac{€}{\text{hora}} \cdot 200 \text{ horas} = 4\,000\,€$$

En el mantenimiento se incluye corrección de errores, adaptación a cambios en las fuentes de obtención de datos (como las modificaciones en la *web* de contratación pública) y actualización de dependencias. A lo largo del proyecto se han producido varias modificaciones del código motivadas por cambios externos, por lo cual ha sido necesario dedicar tiempo periódico al mantenimiento correctivo y evolutivo. Este coste se traduce en un 5 % del

tiempo total de desarrollo del proyecto. Por lo cual el coste de mantenimiento ha sido:

$$12\,614,62\text{ €} \cdot 0,05 = 630,73\text{ €}$$

Respecto a la infraestructura necesaria para el proyecto, se basa en servicios de red, bases de datos y servidores locales. Se han utilizado infraestructuras universitarias (el servidor cuya amortización ya se ha tenido en cuenta en el apartado de adquisición de *hardware*) y servicios gratuitos, por lo cual no es necesario sumar gastos e infraestructuras al proyecto.

El tiempo inactivo se corresponde al impacto económico que suponen los periodos en los que el sistema no está operativo o no puede utilizarse de forma eficiente.

Finalmente, deben considerarse los costes asociados al espacio físico del trabajo y al consumo de energía del equipo. Se ha considerado que para llevar a cabo el proyecto se ha gastado una media de 80 € mensuales de electricidad, como el proyecto ha durado 9 meses, el coste total sería:

$$80 \frac{\text{€}}{\text{meses}} \cdot 9 \text{ meses} = 720\text{ €}$$

Los costes totales del proyecto vienen desglosados en la tabla [A.3](#).

Componentes	Estimación (euros)
Adquisición <i>hardware</i>	180
Adquisición <i>software</i>	21,75
Instalación	2 522,92
Capacitación	3 153,66
Soporte	4 000
Mantenimiento	630,73
Infraestructura	0
Tiempo inactivo	0
Espacio y energía	720
Coste total del proyecto	11 229

Tabla A.3: Coste total del proyecto.

Viabilidad legal

Apéndice B

Especificación de Requisitos

B.1. Introducción

B.2. Objetivos generales

Los objetivos generales que se buscan cumplir con este proyecto son:

- Desarrollar un sistema basado en Generación Aumentada por Recuperación (*RAG*) que permita realizar consultas sobre licitaciones del sector público, enriqueciendo las respuestas mediante información documental relevante.
- Automatizar la recopilación, procesamiento e indexación de documentos de licitaciones, de forma que puedan ser integrados eficientemente en una base de datos vectorial.
- Facilitar la consulta de la información de las licitaciones mediante una aplicación web intuitiva y accesible para usuarios autenticados.
- Mejorar la comprensión y análisis de la información administrativa y técnica contenida en las licitaciones.

B.3. Catálogo de requisitos

A continuación se van a definir los requisitos del proyecto, tanto los funcionales como los no funcionales.

Requisitos funcionales

- **RF-1** Gestión de documentos: el sistema debe permitir que el administrador gestione los documentos de licitaciones del sistema.
 - **RF-1.1** Inserción de documentos: el sistema debe permitir cargar documentos de licitaciones con el objetivo de construir y actualizar la base de datos vectorial.
 - **RF-1.2** Listado de documentos cargados: el sistema debe permitir ver los documentos cargados y sus metadatos.
 - **RF-1.3** Borrado de documentos: el sistema debe permitir borrar los documentos cargados.
 - **RF-1.4** Descarga de documentos: el sistema debe permitir descargar los documentos cargados.
- **RF-2** Procesado de los documentos: el sistema debe procesar de manera automática los documentos introducidos para prepararlos para que el *RAG* pueda usarlos.
 - **RF-2.1** Extracción de información: el sistema debe extraer automáticamente el contenido textual de los documentos introducidos, ignorando páginas o documentos sin texto extraíble.
 - **RF-2.2** Conversión del contenido: el sistema debe transformar el texto extraído a un formato homogéneo, concretamente a *markdown* para que sea más fácil su procesamiento.
 - **RF-2.2.1** Conservar estructura básica: el sistema debe conservar la estructura básica del texto siempre que sea posible (secciones, títulos, saltos de línea, párrafos, etc).
 - **RF-2.3** Limpieza y normalización del texto: el sistema debe limpiar y normalizar el texto que se extrae de los documentos.
 - **RF-2.3.1** Eliminar caracteres: el sistema debe eliminar los caracteres irrelevantes o problemáticos.
 - **RF-2.3.2** Normalización: el sistema debe normalizar los espacios, saltos de línea y formatos inconsistentes.
- **RF-3** Segmentación en *chunks*: el sistema debe dividir los documentos ya procesados en fragmentos de texto (*chunks*) adecuados para el modelo de *embeddings*.
 - **RF-3.1** Tamaño: el sistema debe segmentar el texto respetando un máximo de tamaño medido en número de *tokens* para evitar errores.
 - **RF-3.2** Solapamiento: el sistema debe aplicar solapamiento entre *chunks* para preservar el contexto.
 - **RF-3.3** Contexto: cada *chunk* debe tener una referencia al documento original y a su posición dentro de él.

- **RF-4** Generación de *embeddings*: el sistema debe generar representaciones vectoriales (*embeddings*) para cada *chunk*.
- **RF-5** Persistencia en base de datos vectorial: el sistema debe almacenar los *chunks* y sus *embeddings* en una base de datos vectorial.
 - **RF-5.1** Almacenar *chunks*: el sistema debe almacenar el contenido textual de cada *chunk*.
 - **RF-5.2** Almacenar *embeddings*: el sistema debe almacenar el *embedding* asociado a cada *chunk*.
 - **RF-5.3** Almacenar metadatos: el sistema debe almacenar metadatos relevantes como el nombre del archivo, el título y el segmento incluido en el *chunk*.
 - **RF-5.4** Actualización: el sistema debe permitir la actualización o recreación de la colección vectorial en el caso de incluir documentos nuevos.
 - **RF-5.5** Eliminar: el sistema debe permitir borrar los *chunks* y *embeddings* de la base de datos vectorial.
- **RF-6** Recuperación de información por similitud: el sistema debe recuperar información relevante a partir de las consultas de los usuarios.
 - **RF-6.1** Generación de *embedding*: el sistema debe generar el *embedding* de la consulta del usuario.
 - **RF-6.2** Búsqueda por similitud: el sistema debe realizar una búsqueda por similitud en la base de datos vectorial.
- **RF-7** Construcción del *prompt* con contexto: el sistema debe construir un *prompt*, para alimentar al *LLM*, que incluya los *chunks* recuperados, la pregunta del usuario e instrucciones de uso del contexto.
- **RF-8** Generación de respuestas mediante un *LLM*: el sistema debe enviar el *prompt* generado a un modelo *LLM* y mostrar la respuesta de dicho modelo al usuario.
- **RF-9** Trazabilidad de respuesta: el sistema debe devolver junto con la respuesta los metadatos del documento origen y los *chunks* empleados para la generación de la respuesta.
- **RF-10** Gestión de usuarios: el sistema debe permitir la gestión de usuarios a través de la aplicación web.
 - **RF-10.1** Registros y autenticación: el sistema debe permitir el registro y autenticación de los usuarios.
 - **RF-10.2** Sesiones: el sistema debe gestionar las sesiones de los usuarios de forma segura.
 - **RF-10.3** Roles: el sistema debe permitir asignar a los usuarios el rol de administradores o usuarios normales.

- **RF-10.4** Administradores: los administradores deben poder añadir usuarios, eliminarlos y cambiar su rol.
- **RF-10.5** Edición de datos de usuario: los usuarios deben poder editar sus datos.
- **RF-10.6** Recuperación de contraseña: los usuarios deben poder cambiar su contraseña utilizando su correo.
- **RF-11** Interfaz de consultas: los usuarios autenticados podrán realizar consultas y ver las respuestas del sistema.
 - **RF-11.1** Historial de consultas: los usuarios podrán ver su historial de consultas.
 - **RF-11.2** Borrar consultas: los usuarios podrán borrar consultas del historial.
- **RF-12** Importación automática de licitaciones: el sistema debe permitir al administrador importar de manera automática las licitaciones mediante *web scraping*.
 - **RF-12.1** Descarga y almacenamiento: el sistema debe descargar y almacenar los documentos obtenidos, así como sus metadatos.
 - **RF-12.2** Gestión de errores y duplicados: el sistema debe continuar aunque la descarga de algún documento falle. Debe registrar los errores y evitar duplicados.

Requisitos no funcionales

- **RNF-1** Reproducibilidad: el proyecto debe ejecutarse en un entorno reproducible.
- **RNF-2** Rendimiento: el sistema debe ser capaz de procesar los documentos sin intervención manual. Además, el sistema tiene que responder en un tiempo razonable.
- **RNF-3** Robustez ante errores: el sistema debe continuar ejecutándose ante fallos esperables en datos, red o servicios externos.
 - **RNF-3.1** Documentos corruptos: el sistema debe saltarse los documentos corruptos o sin texto extraíble y continuar con el resto de documentos.
 - **RNF-3.2** *Timeouts*: el sistema debe manejar los *timeouts* del *LLM* devolviendo mensajes controlados al usuario.
- **RNF-4** Trazabilidad técnica: el sistema debe realizar *logs* de carga de modelo, lectura de los PDF, *chunking*, *embeddings*, guardado y consulta.

- **RNF-5** Seguridad: el sistema debe gestionar de forma segura la información sensible de los usuarios y el acceso a la aplicación.
 - **RNF-5.1** Contraseñas: las contraseñas deben almacenarse encriptadas.
 - **RNF-5.2** Control de acceso por roles: las funciones administrativas deben estar protegidas por rol para que solo las pueda realizar el administrador.
 - **RNF-5.3** Protección de secretos: las claves sensibles no deben estar públicas en el repositorio.
- **RNF-6** Usabilidad: la interfaz debe permitir a los usuarios realizar las consultas y ver el historial de manera sencilla. Además, los textos deben ser legibles.
- **RNF-7** Calidad del *software*: debe existir un conjunto de pruebas que asegure el correcto funcionamiento del programa.
- **RNF-8** Disponibilidad: la web debe garantizar un nivel de disponibilidad adecuado que permita a los usuarios acceder a la aplicación y realizar consultas.
- **RNF-9** Recuperación y tolerancia a fallos: el sistema debe disponer de mecanismos que permitan la recuperación de información y la restauración del servicio tras fallos, garantizando la integridad de los datos.
- **RNF-10** Evitar alucinaciones: el sistema no debe responder cuando no haya contexto relevante suficiente.
- **RNF-11** Calidad de las respuestas del *LLM*: el sistema debe llevar a cabo un control de calidad de las respuestas que da el *LLM* a las consultas del usuario.

B.4. Especificación de requisitos

Los actores del sistema son:

- **Usuario anónimo**: usuario que no ha iniciado sesión (solo puede registrarse y autenticarse)
- **Usuario autenticado**: usuario que puede acceder a la aplicación sus acciones en al aplicación cambiarán dependiendo de su rol.
 - **Usuario normal**: usuario autenticado que puede consultar el *RAG* y ver su historial
 - **Usuario administrador**: usuario autenticado con permisos de gestión de usuarios y de carga y actualización documental

- **Sistema:** es el encargado del procesamiento, *chunking*, *embeddings*, guardado y consulta.

Los casos de uso definidos para el proyecto son:

- **CU-1** Registro de usuarios (ver tabla [B.1](#)).
- **CU-2** Iniciar sesión (ver tabla [B.2](#)).
- **CU-3** Cerrar sesión (ver tabla [B.3](#)).
- **CU-4** Recuperar contraseña (ver tabla [B.4](#)).
- **CU-5** Realizar consulta al *RAG* (ver tabla [B.5](#)).
- **CU-6** Consultar historial de consultas (ver tabla [B.6](#)).
- **CU-7** Borrar consultas del historial (ver tabla [B.7](#)).
- **CU-8** Gestión de documentos (ver tabla [B.8](#)).
- **CU-9** Cargar documentos de licitaciones (ver tabla [B.9](#)).
- **CU-10** Importar licitaciones automáticamente (ver tabla [B.10](#)).
- **CU-11** Convertir los documentos a *Markdown* (ver tabla [B.11](#)).
- **CU-12** Indexar documento (ver tabla [B.12](#)).
- **CU-13** Listar documentos (ver tabla [B.13](#)).
- **CU-14** Consultar estado del documento (ver tabla [B.14](#)).
- **CU-15** Eliminar documento (ver tabla [B.15](#)).
- **CU-16** Descargar documento (ver tabla [B.16](#)).
- **CU-17** Gestión de usuario (ver tabla [B.17](#)).
- **CU-18** Crear usuario (ver tabla [B.18](#)).
- **CU-19** Borrar usuario (ver tabla [B.19](#)).
- **CU-20** Modificar rol del usuario (ver tabla [B.20](#)).
- **CU-21** Editar usuario (ver tabla [B.21](#)).

El diagrama de los casos de uso se muestra en la imagen [B.1](#).

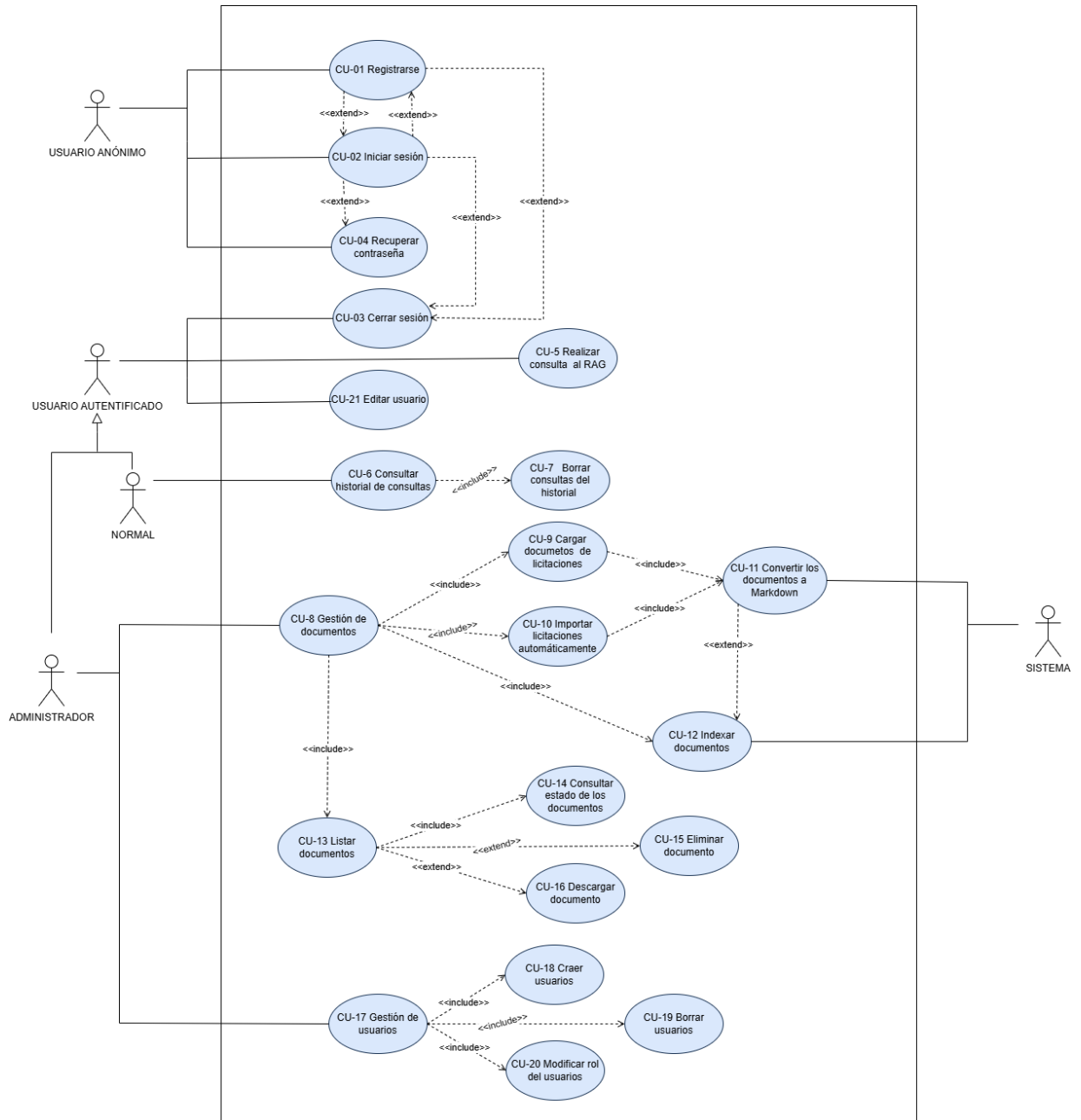


Figura B.1: Diagrama general de casos de uso.

CU-1	Registro de usuarios
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.1
Descripción	Permite crear una cuenta en la aplicación web.
Precondición	Que el usuario no esté autenticado.
Acciones	1. El usuario accede a «Registro». 2. El usuario introduce el nombre, <i>email</i> y contraseña. 3. El sistema valida el formato y que el <i>email</i> no esté duplicado. 4. El sistema almacena el usuario con la contraseña encriptada. 5. El sistema confirma el registro. 6. El sistema crea la sesión y redirige a la página de consultas.
Postcondición	Cuenta creada y usuario autenticado
Excepciones	1. Tiene algún campo vacío $\Rightarrow$ Mostrar error de campo obligatorio. 2. El <i>email</i> ya existe $\Rightarrow$ Mostrar error de <i>email</i> duplicado. 3. El <i>email</i> no cumple la validación $\Rightarrow$ Mostrar error de <i>email</i> inválido. 4. La contraseña no cumple la política $\Rightarrow$ Mostrar error de contraseña poco segura. 5. Las contraseñas no coinciden $\Rightarrow$ Mostrar error de contraseñas distintas. 6. El nombre tiene menos de 2 caracteres $\Rightarrow$ Mostrar error de nombre demasiado corto.
Importancia	Alta.

Tabla B.1: CU-1 Registro de usuarios.

CU-2	Iniciar sesión
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.1 , RF-10.2
Descripción	Autentifica al usuario y abre una sesión segura.
Precondición	Que el usuario esté registrado y no esté autenticado.
Acciones	1. El usuario accede a «Iniciar sesión». 2. El usuario introduce el <i>email</i> y la contraseña. 3. El sistema verifica los credenciales. 4. El sistema crea la sesión y redirige a la página de consultas.
Postcondición	Sesión activa para el usuario.
Excepciones	1. Tiene algún campo vacío $\Rightarrow$ Mostrar error de campo obligatorio. 2. El <i>email</i> o la contraseña no coinciden con ningún usuario registrado $\Rightarrow$ Mostrar error de usuario o contraseña incorrectos. 3. El <i>email</i> no cumple la validación $\Rightarrow$ Mostrar error de <i>email</i> inválido. 4. La contraseña no cumple la política $\Rightarrow$ Mostrar error de contraseña poco segura.
Importancia	Alta.

Tabla B.2: CU-2 Iniciar sesión.

CU-3	Cerrar sesión
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.2
Descripción	Finaliza la sesión del usuario.
Precondición	Que el usuario esté autenticado.
Acciones	1. El usuario pulsa el botón «Cerrar sesión». 2. El sistema invalida la sesión. 3. El sistema redirige a la página de inicio.
Postcondición	Sesión finalizada.
Excepciones	1. Error al invalidar sesión $\Rightarrow$ Mostrar mensaje de error al cerrar sesión.
Importancia	Media.

Tabla B.3: CU-3 Cerrar sesión.

CU-4	Recuperar contraseña
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10 , RF-10.1 , RF-10.6
Descripción	Permite a los usuarios cambiar su contraseña antes de iniciar sesión.
Precondición	Que el usuario esté registrado en el sistema.
Acciones	1. El usuario accede a «Iniciar sesión». 2. El usuario pulsa el enlace «¿No recuerdas la contraseña?». 3. El sistema envía un correo al email del usuario con el enlace a la página de recuperación de contraseña. 4. El usuario accede a la página de recuperar contraseña. 5. El usuario introduce la nueva contraseña y la confirmación de la contraseña. 6. El sistema guarda la nueva contraseña encriptada.
Postcondición	Contraseña cambiada para el usuario.
Excepciones	1. Error al enviar el correo $\Rightarrow$ Mostrar error al enviar el correo. 2. La contraseña no cumple la política $\Rightarrow$ Mostrar error de contraseña poco segura. 3. Las contraseñas no coinciden $\Rightarrow$ Mostrar error de contraseñas distintas.
Importancia	Media.

Tabla B.4: CU-4 Recuperar contraseña.

CU-5	Realizar consulta al <i>RAG</i>
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-6 , RF-7 , RF-8 , RF-9 , RF-11 , RNF-3 , RNF-3.2 , RNF-10 , RNF-11
Descripción	El usuario hace una pregunta, el sistema recupera contexto por similitud, construye el <i>prompt</i> y genera respuesta que incluya las fuentes.
Precondición	Que el usuario esté autenticado y la base de datos vectorial esté disponible.
Acciones	1. El usuario escribe y envía una pregunta. 2. El sistema genera <i>embeddings</i> de la consulta. 3. El sistema busca por similitud en la base de datos vectorial. 4. El sistema construye un <i>prompt</i> con los <i>chunks</i> y las reglas de uso del contexto. 5. El sistema llama al <i>LLM</i> y genera respuestas. 6. El sistema adjunta la trazabilidad de la respuesta, la cual incluye los documentos, metadatos y <i>chunks</i> utilizados. 7. El sistema guarda la consulta en el historial del consultas del usuario.
Postcondición	Respuesta mostrada y consulta registrada.
Excepciones	1. Cuando no hay contexto suficiente $\Rightarrow$ El sistema no responde a la pregunta (para evitar alucinaciones) y muestra un aviso de que falta contexto. 2. <i>Timeout</i> del <i>LLM</i> $\Rightarrow$ Mostrar un mensaje controlado.
Importancia	Alta.

Tabla B.5: CU-5 Realizar consulta al *RAG*.

CU-6	Consultar historial de consultas
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-11.1
Descripción	Permite al usuario ver su lista de preguntas y respuestas anteriores.
Precondición	Que el usuario esté autenticado y existan consultas guardadas.
Acciones	1. El usuario accede al «Historial». 2. El sistema lista las consultas del usuario. 3. El usuario puede abrir una consulta para ver la respuesta completa y la trazabilidad de dicha respuesta.
Postcondición	Historial visualizado.
Excepciones	1. El usuario no ha realizado consultas $\Rightarrow$ Mostrar mensaje de que aun no hay consultas.
Importancia	Alta.

Tabla B.6: **CU-6** Consultar historial de consultas.

CU-7	Borrar consultas del historial
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-11.2
Descripción	Permite al usuario borrar consultas del historial.
Precondición	Que el usuario esté autenticado y existan consultas guardadas.
Acciones	1. El usuario accede al «Historial». 2. El sistema lista las consultas del usuario. 3. El usuario pulsa el botón «Borrar». 4. El sistema elimina la consulta seleccionada de la base de datos. 5. El sistema actualiza la vista del historial.
Postcondición	La consulta seleccionada se elimina del historial del usuario.
Excepciones	1. Error en la base de datos al intentar borrar la consulta ⇒ Mostrar mensaje de error controlado. 2. La consulta seleccionada no existe o ya ha sido borrada ⇒ Mostrar aviso al usuario.
Importancia	Alta.

Tabla B.7: CU-7 Borrar consultas del historial.

CU-8	Gestión de documentos
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1 , RF-1.1 , RF-1.2 , RF-1.3 , RF-2 , RF-2.1 , RF-2.2 , RF-2.2.1 , RF-2.3 , RF-2.3.1 , RF-2.3.2 , RF-3 , RF-4 , RF-5 , RF-5.1 , RF-5.2 , RF-5.3 , RF-5.4 , RF-5.5 , RNF-3.1
Descripción	El administrador debe poder gestionar a los documentos de la aplicación.
Precondición	Hay un administrador autenticado.
Acciones	1. El administrador accede a la página de «Iniciar sesión». 2. El administrador introduce su <i>email</i> y contraseña. 3. El sistema le autentifica como administrador. 4. El administrador accede a «Administrar documentos».
Postcondición	Posibilidad de cargar documentos nuevos (de forma manual o por <i>web scraping</i>), indexar, borrar o descargar los documentos ya cargados .
Excepciones	1. Intento de acceso sin ser administrador $\Rightarrow$ Se deniega. 2. Si no hay documentos cargados en la aplicación $\Rightarrow$ Muestra un mensaje.
Importancia	Alta.

Tabla B.8: **CU-8** Gestión de documentos.

CU-9	Cargar documentos de licitaciones
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1 , RF-10.3 , RF-10.4 , RNF-3 , RNF-3.1
Descripción	El administrador sube documentos para alimentar o ampliar la base de datos vectorial.
Precondición	Que el administrador esté autenticado.
Acciones	1. El administrador pulsa el botón «Cargar». 2. El administrador selecciona uno o varios documentos. 3. El sistema valida los documentos y los almacena. 4. El sistema registra la carga.
Postcondición	Los documentos están disponibles para procesado.
Excepciones	1. Documento corrupto o sin texto legible $\Rightarrow$ Se marca como fallido y se continua con el siguiente.
Importancia	Alta.

Tabla B.9: **CU-9** Cargar documentos de licitaciones.

CU-10	Importar licitaciones automáticamente
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-12 , RF-12.1 , RF-12.2 , RNF-3 , RNF-3.1
Descripción	Permite al administrador importar de manera automática las licitaciones a través de <i>web scraping</i> .
Precondición	Que el administrador esté autenticado y haya un programa de <i>web scraping</i> funcional.
Acciones	1. El administrador accede a «Administrar documentos». 2. El administrador pulsa el botón «Web scraping». 3. El sistema lanza el proceso de <i>web scraping</i>. 4. El sistema guarda los documentos obtenidos y registra sus metadatos. 5. El sistema muestra un resumen con el número de documentos descargados, los duplicados y los fallidos.
Postcondición	Los documentos importados están disponibles para procesado e indexado.
Excepciones	1. Fallo de red $\Rightarrow$ Se registra y muestra al administrador. 2. Documento corrupto o sin texto legible $\Rightarrow$ Se marca como fallido y se continua con el siguiente. 3. Documento duplicado $\Rightarrow$ Se omite y se registra como duplicado.
Importancia	Media.

Tabla B.10: **CU-10** Importar licitaciones automáticamente.

CU-11	Convertir los documentos a <i>Markdown</i>
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-2 , RF-2.1 , RF-2.2 , RF-2.2.1 , RF-2.3 , RF-2.3.1 , RF-2.3.2
Descripción	El sistema prepara automáticamente los documentos para que puedan ser procesados por el sistema.
Precondición	Hay documentos cargados por el administrador.
Acciones	1. El sistema extrae texto descartando páginas y documentos sin texto extraíble. 2. Convierte el texto a <i>Markdown</i> homogéneo. 3. Intenta conservar la estructura básica del documento original. 4. Limpia caracteres problemáticos y normaliza espacios y saltos de línea.
Postcondición	Los documentos convertidos a <i>Markdown</i> listos para el <i>chunking</i> .
Excepciones	1. Documento sin texto $\Rightarrow$ Se marca como no procesable y se continua como los demás.
Importancia	Alta.

Tabla B.11: CU-11 Convertir los documentos a *Markdown*.

CU-12	Indexar documento
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-3 , RF-4 , RF-5 , RF-5.1 , RF-5.2 , RF-5.3
Descripción	Permite al administrador actualizar la base de datos vectorial cuando hay documentos nuevos. El sistema convierte los documentos procesados en <i>chunks</i> , genera los <i>embeddings</i> y los guarda en la base de datos vectorial con metadatos.
Precondición	Existen documentos procesados, una base de datos vectorial y un administrador autenticado.
Acciones	1. El administrador accede a «Administrar documentos». 2. El administrador pulsa el botón «Actualizar Base de Datos Vectorial». 3. El sistema segmenta los archivos en <i>chunks</i>. 4. Aplica solapamiento a dichos <i>chunks</i>. 5. Añade una referencia al documento así como el segmento que contiene. 6. Genera <i>embeddings</i> de cada <i>chunk</i>. 7. Lo almacena en la base de datos vectorial.
Postcondición	La colección vectorial con los nuevos documentos incluidos.
Excepciones	1. Fallo en conseguir los <i>embedding</i> de un <i>chunk</i> $\Rightarrow$ Se omite el <i>chunk</i> y se continúa. 2. Error de conexión con la base de datos vectorial $\Rightarrow$ Se aborta la operación y se avisa al administrador a través de un mensaje.
Importancia	Alta.

Tabla B.12: CU-12 Indexar documento.

CU-13	Listar documentos
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1.2 , RNF-4
Descripción	Permite al administrador ver el listado de documentos junto con los metadatos relevantes.
Precondición	Hay un administrador autenticado y existe al menos un documento cargado en el sistema.
Acciones	1. El administrador accede a «Administrar documentos». 2. El sistema lista los documentos con sus metadatos (nombre, tamaño, fecha de modificación y <i>chunks</i>).
Postcondición	Documentos visibles para el administrador.
Excepciones	1. No hay documentos $\Rightarrow$ Muestra un mensaje informativo «Aún no hay documentos». 2. Error de base de datos $\Rightarrow$ Muestra un error.
Importancia	Media.

Tabla B.13: CU-13 Listar documentos.

CU-14	Consultar estado del documento
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1.2 , RNF-4
Descripción	Permite al administrador ver el estado de los documentos cargados en el sistema (cargado, indexado o fallido).
Precondición	Hay un administrador autenticado y existe al menos un documento cargado en el sistema.
Acciones	1. El administrador accede a «Administrar documentos». 2. El sistema lista los documentos con sus metadatos (nombre, tamaño, fecha de modificación y <i>chunks</i>). 3. El sistema muestra el estado de los documentos listados.
Postcondición	Estados de los documentos visibles para el administrador.
Excepciones	1. No hay documentos $\Rightarrow$ Muestra un mensaje informativo «Aún no hay documentos». 2. Error de base de datos $\Rightarrow$ Muestra un error.
Importancia	Media.

Tabla B.14: **CU-14** Consultar estado del documento.

CU-15	Eliminar documento
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1.3 , RF-5.5 , RNF-5.2
Descripción	Permite al administrador eliminar un documento del sistema, así como sus <i>chunks</i> y <i>embeddings</i> asociados en la base de datos vectorial.
Precondición	Hay un administrador autenticado y existe al menos un documento cargado en el sistema.
Acciones	1. El administrador accede a «Administrar documentos». 2. El sistema lista los documentos con sus metadatos (nombre, tamaño, fecha de modificación y <i>chunks</i>) y estado. 3. El administrador pulsa el botón «Borrar». 4. El sistema pide confirmación. 5. El administrador confirma el borrado. 6. El sistema borra el documento y sus metadatos. Además, si el documento esta indexado borra sus <i>chunks</i> y sus <i>embeddings</i>. 7. El sistema actualiza el listado.
Postcondición	Documento eliminado y sus <i>chunks</i> y <i>embeddings</i> borrados de la base de datos vectorial.
Excepciones	1. El documento no existe $\Rightarrow$ Se muestra un aviso. 2. Error al borra $\Rightarrow$ No se borra y se informa al administrador. 3. Error al borrar en la base de datos vectorial $\Rightarrow$ No se borra el documento y se informa al administrador.
Importancia	Alta.

Tabla B.15: CU-15 Eliminar documento.

CU-16	Descargar documento
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-1 , RF-1.4 , RNF-5.2
Descripción	Permite al administrador descargar un documento del sistema.
Precondición	Hay un administrador autenticado y existe al menos un documento cargado en el sistema.
Acciones	1. El administrador accede a «Administrar documentos». 2. El sistema lista los documentos con sus metadatos (nombre, tamaño, fecha de modificación y <i>chunks</i>) y estado. 3. El administrador pulsa el botón «Descargar». 4. Se descarga el documento.
Postcondición	Documento descargado en el equipo del administrador.
Excepciones	1. El documento no existe $\Rightarrow$ Se muestra un aviso. 2. Error al descargar $\Rightarrow$ Se informa al administrador.
Importancia	Media.

Tabla B.16: CU-16 Descargar documento.

CU-17	Gestión de usuarios
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3 , RF-10.4
Descripción	El administrador debe poder gestionar a los usuarios de la aplicación y sus permisos.
Precondición	Hay un administrador autenticado.
Acciones	1. El administrador accede a la página de «Iniciar sesión». 2. El administrador introduce su <i>email</i> y contraseña. 3. El sistema le autentifica como administrador. 4. El administrador accede a «Administrar usuarios».
Postcondición	Listado de usuarios para editar.
Excepciones	1. Intento de acceso sin ser administrador $\Rightarrow$ Se deniega. 2. Si no hay usuarios registrados en la aplicación $\Rightarrow$ Mostrar mensaje de que aún no hay usuarios.
Importancia	Media.

Tabla B.17: CU-17 Gestión de usuarios.

CU-18	Crear usuario
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3 , RF-10.4
Descripción	El administrador crea usuarios desde la web.
Precondición	Hay un administrador autenticado.
Acciones	1. El administrador accede a «Administrar usuarios». 2. El administrador pulsa el botón «Añadir usuario». 3. El administrador introduce el nombre, <i>email</i>, contraseña y si es administrador o no. 4. El sistema valida el formato y que el <i>email</i> no esté duplicado. 5. El sistema almacena el usuario con la contraseña encriptada. 6. El sistema redirige a la página de «Gestión de usuarios».
Postcondición	Usuario creado.
Excepciones	1. Intento de acceso sin ser administrador $\Rightarrow$ Se deniega. 2. Tiene algún campo vacío $\Rightarrow$ Muestra un error con el mensaje «Completa este campo». 3. El <i>email</i> ya existe $\Rightarrow$ Muestra un error con el mensaje «Ya existe un usuario con ese <i>email</i>». 4. El <i>email</i> no cumple la validación $\Rightarrow$ Muestra un error con el mensaje «Invalid <i>email</i> address». 5. La contraseña no cumple la política $\Rightarrow$ Muestra un error con el mensaje «Aumenta la longitud del texto a 6 caracteres como mínimo». 6. El nombre tiene menos de 2 caracteres $\Rightarrow$ Muestra un error con el mensaje «Aumenta la longitud del texto a 2 caracteres o más».
Importancia	Media.

Tabla B.18: CU-18 Crear usuario.

CU-19	Borrar usuario
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3 , RF-10.4
Descripción	El administrador borra usuarios desde la web.
Precondición	Hay un administrador autenticado y usuarios previamente registrados en la aplicación.
Acciones	1. El administrador accede a «Administrar usuarios». 2. El administrador pulsa el botón «Borrar» del usuario que desee eliminar. 3. El sistema pide confirmación antes de borrar el usuario. 4. El administrador pulsa el botón «Aceptar». 5. El sistema borra al usuario y sus datos de la base de datos.
Postcondición	Usuario borrado.
Excepciones	1. Fallo al conectar con la base de datos $\Rightarrow$ No se borra el usuario y se informa del problema al administrador.
Importancia	Media.

Tabla B.19: CU-19 Borrar usuario.

CU-20	Modificar rol del usuario
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.3 , RF-10.4
Descripción	El administrador puede cambiar el rol de los usuarios desde la web.
Precondición	Hay un administrador autenticado y usuarios previamente registrados en la aplicación.
Acciones	1. El administrador accede a «Administrar usuarios». 2. El administrador pulsa el botón «Hacer admin»/«Quitar admin» del usuario que desee cambiar de rol. 3. El sistema cambia el rol del usuario.
Postcondición	Usuario con el rol cambiado.
Excepciones	1. Fallo al conectar con la base de datos $\Rightarrow$ No se cambia el rol del usuario y se informa del problema al administrador.
Importancia	Baja.

Tabla B.20: **CU-20** Modificar rol del usuario.

CU-21	Editar usuario
Versión	1.0
Autor	Lydia Blanco Ruiz
Requisitos asociados	RF-10.5 , RNF-5.1 , RNF-5.2
Descripción	Permite que un usuario edite sus datos y contraseña.
Precondición	Hay un usuario autenticado.
Acciones	1. El usuario accede a «Editar usuario». 2. Modifica los campos que desee (nombre, <i>email</i> o contraseña). 3. El sistema valida los formatos y reglas. 4. El sistema guarda los cambios. Si hay cambio de contraseña la almacena encriptada.
Postcondición	Datos del usuario actualizados.
Excepciones	1. Se intenta cambiar el <i>email</i> por otro que ya existe en el sistema $\Rightarrow$ Muestra un error con el mensaje «Ya existe un usuario con ese <i>email</i>». 2. El <i>email</i> no cumple la validación $\Rightarrow$ Muestra un error con el mensaje «Invalid <i>email</i> address». 3. La contraseña no cumple la política $\Rightarrow$ Muestra un error con el mensaje «Aumenta la longitud del texto a 6 caracteres como mínimo». 4. El nombre tiene menos de 2 caracteres $\Rightarrow$ Muestra un error con el mensaje «Aumenta la longitud del texto a 2 caracteres o más».
Importancia	Baja.

Tabla B.21: CU-21 Editar usuario.

Apéndice C

Especificación de diseño

C.1. Introducción

C.2. Diseño de datos

Para llevar a cabo la aplicación se han definido las siguientes entidades:

- **Usuario:** esta entidad representa a la persona que usa la aplicación. Distingue entre usuario normal y administrador, al cual se le permite realizar la gestión de documentos y usuarios. Esta entidad maneja un identificador (id) que es la clave primaria, un nombre, un email que debe ser único, una contraseña que se almacena encriptada, el último inicio de sesión y el rol.
- **Consulta:** esta entidad guarda las preguntas que realizan los usuarios, así como las respuestas que da el *LLM* alimentado por el *RAG*. Es la base para el historial de consultas donde se van a listar y borrar. Esta entidad maneja un identificador (id) que es la clave primaria, el identificador del usuario que realiza la consulta, la pregunta del usuario, la respuesta, los fragmentos utilizados, el tiempo de respuesta, y la fecha en la que se preguntó al modelo.
- **Documento:** esta entidad representa un PDF cargado en la aplicación. Maneja un identificador (id) como clave primaria, el nombre del fichero, el tamaño, la fecha de modificación, el número de *chunks* en la base de datos vectorial, el *hash* del documento y el estado del documento (cargado, procesado, indexado, fallido).
- **Chunk:** esta entidad representa cada trozo de texto en el que se dividen los documentos para poder generar los *embeddings*. Maneja el texto

que forma al propio *chunk*, el *embedding* y los metadatos del *chunk* (nombre del archivo, título y posición dentro del documento).

- *Embedding*: esta entidad se corresponde con la representación vectorial utilizada para la base de datos vectorial. Va a permitir recuperar *chunks* por similitud. La entidad maneja un vector asociado al *chunk*.
- *ConsultaChunk*: esta entidad modela la relación entre las consultas y los *chunks* permitiendo que una consulta tenga varios *chunk* y que un *chunk* pueda servir para varias consultas. La entidad maneja el identificador de la consulta, el identificador del *chunk*, la similitud con la consulta del usuario y el *ranking*. El *ranking* hace referencia a la relevancia que tiene un *chunk* entre los recuperados para aumentar el *prompt* del *LLM*. La posición del *chunk* en el *ranking* se determina ordenándolos descendientemente según su similitud.

Las relaciones entre las entidades son:

- Un usuario puede hacer cero o varias consultas.
- Cada consulta pertenece a un usuario.
- Un documento tiene cero (si está cargado pero no indexado) o varios *chunks*.
- Cada *chunk* pertenece a un documento.
- Un *chunk* tiene un *embedding*.
- Cada *embedding* pertenece a un *chunk*.
- Una consulta usa uno o varios *chunk*.
- Un *chunk* puede ser usado por cero o varias consultas.

El diagrama entidad relación se muestra en la imagen [C.1](#) y el diagrama relacional se muestra en la imagen [C.2](#)

C.3. Diseño arquitectónico

El objetivo del diseño arquitectónico es garantizar la modularidad, la escalabilidad, la mantenibilidad, la portabilidad y la reproducibilidad del sistema desarrollado. Es decir, se busca tener una separación clara de responsabilidades entre la interfaz, la lógica, la persistencia y los servicios de Inteligencia Artificial con una organización del código en componentes cohesivos (*Blueprints*, servicios y modelos), pudiendo replicarse los servicios de manera sencilla. Además, sustituir los componentes debe tener un impacto mínimo en el resto del código.

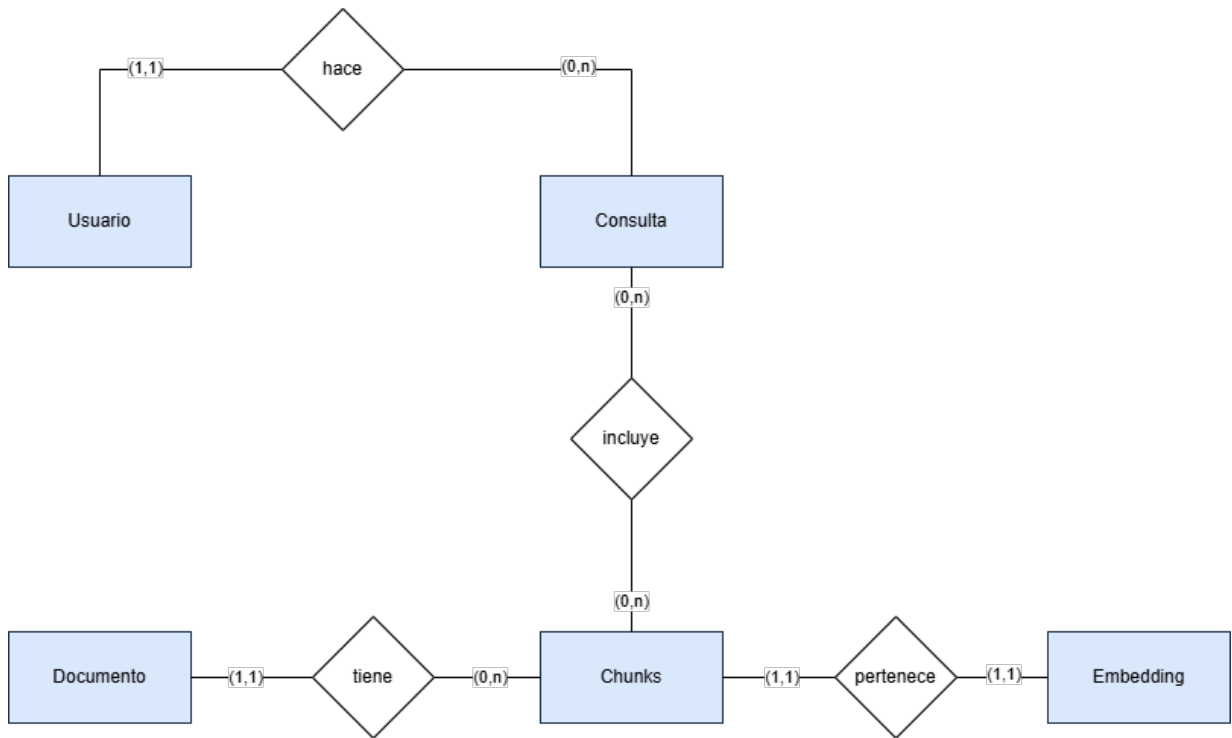


Figura C.1: Diagrama Entidad/Relación.

Este sistema se basa en una arquitectura cliente-servidor multicapa, combinada con una arquitectura *RAG* (*Retrieval-Augmented Generation*), en la capa de aplicación, estructurada en dos *pipelines* diferenciados, el de indexación y el de generación. El despliegue se lleva a cabo utilizando contenedores *Docker* para almacenar el código junto con sus dependencias.

Arquitectura general del sistema y despliegue

El sistema sigue una arquitectura distribuida basada en servicios (*service-oriented* [4]), donde cada componente tiene una responsabilidad claramente definida. La aplicación se compone de los siguientes elementos principales:

- Cliente *web*: se corresponde con el navegador del usuario.
- Servidor de *proxy* inverso *Nginx*: el cual se encarga del protocolo de seguridad a través del certificado SSL/TLS, las redirecciones HTTP a HTTPS y el encaminamiento hacia la aplicación .

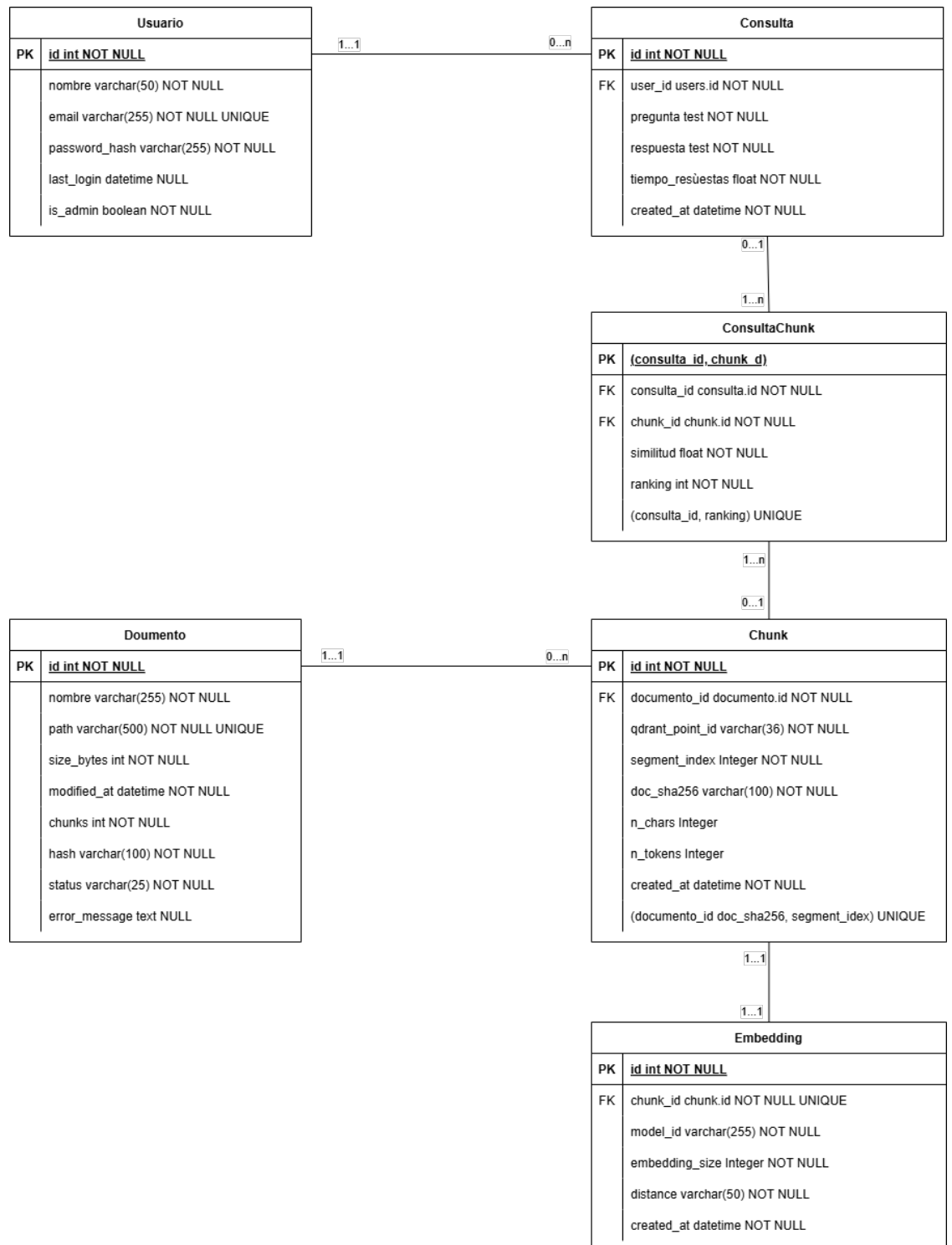


Figura C.2: Diagrama relacional.

- Capa de aplicación: aplicación *Flask* desplegada tras un servidor WSGI *Gunicorn*.
- Persistencia relacional: se incluye *PostgreSQL* como base de datos relacional para asegurar la persistencia de los datos de las transacciones, como son los usuarios, las consultas, los documentos o los estados.
- Persistencia vectorial: se utiliza *Qdrant* como base de datos vectorial para realizar la búsqueda semántica sobre los *embeddings*.
- Servicio de modelos *LLM*: se utiliza *Ollama* para servir el modelo *LLM* de manera local exponiendo una API HTTP de generación.
- Contenedor de pruebas: se utiliza este contenedor para la ejecución de las pruebas del sistema, así como, de la cobertura de dichas pruebas.

El sistema se despliega mediante contenedores *Docker*. Los servicios se definen en el archivo **Docker Compose**. Se definen seis contenedores independientes que se corresponden con el entorno para la base de datos relacional *PostgreSQL* llamado **db**, el de la base de datos vectorial *Qdrant* llamado **qdrant**, el de el servicio *LLM* llamado **Ollama**, el de la aplicación *Flask* levantada usando *Gunicorn* llamado **web**, el del *proxy* inverso **nginx** y el contenedor de ejecución de pruebas llamado **test**.

Gracias a esta separación se garantiza la seguridad ya que no se expone directamente la aplicación *Flask*. Este despliegue también, aumenta la escalabilidad, la reproducibilidad y la portabilidad al tener la posibilidad de replicar los servicios en un entorno aislado que no depende del sistema anfitrión.

La comunicación entre componentes (ver imagen [C.3](#)) se realiza principalmente dentro de al red interna de **Docker-compose**, donde los servicios se resuelven entre sí mediante el nombre de servicio, es decir, el DNS interno, y se conectan a través de los puertos expuestos internamente. El acceso desde el exterior se gestiona usando *Nginx*, que es el único componenete que publica puertos hacia internet (el puerto 80 para el protocolo HTTP y el 443 para el HTTPS). *Nginx* actúa como *proxy* inverso y punto de terminación TLS [\[2\]](#) (TLS es un protocolo que asegura la privacidad y la seguridad de los datos). Recibe las peticiones que hacen los usuarios a la *web* **pythia.es** y las reenvía por HTTP interno al servicio *web* en **web:5000** que se corresponde con la aplicación *Flask* servida con *Gunicorn*. Desde la aplicación, las comunicaciones con los servicios de datos se realizan dentro del entorno *Docker*. La conexión con la base de datos relacional *PostgreSQL* se gestiona con una conexión TCP a **db:5432**. La conexión con la base de datos vectorial *Qdrant*, para la recuperación semántica, se realiza mediante conexión HTTP por el puerto **qdrant:6333**. Para generar las respuestas se

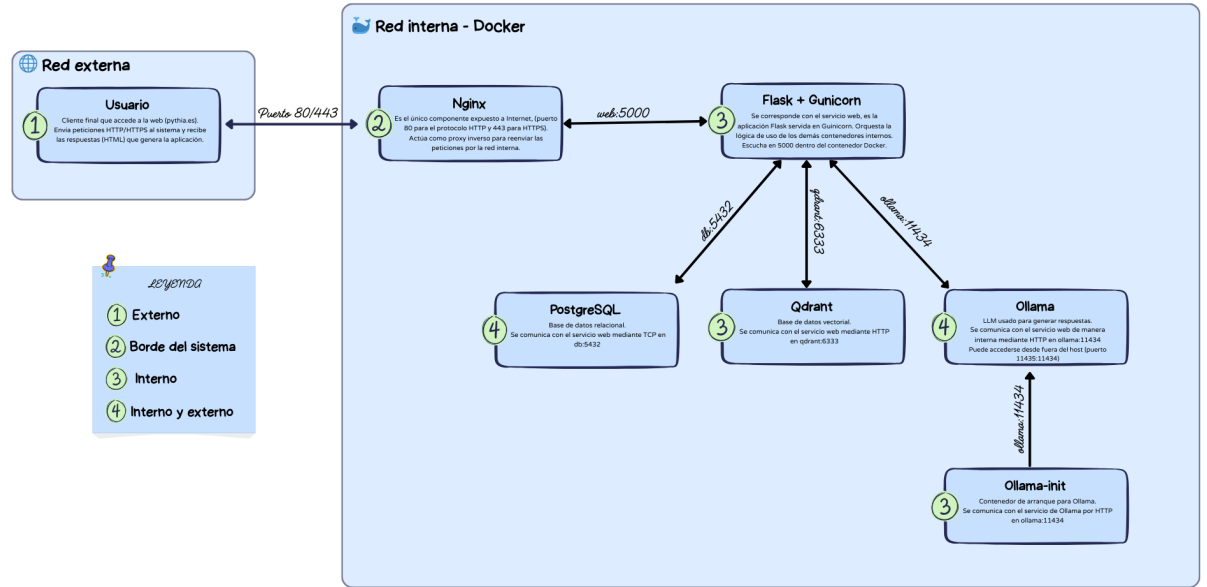


Figura C.3: Diagrama de la comunicación entre los principales componentes del sistema.

invoca el servicio *Ollama* por HTTP en `ollama:11434`. *Ollama* puede ser accesible desde fuera del *host* gracias al mapeo de puertos `11435:11434`. Por último, el contenedor *ollama-init* se comunica con *Ollama* de forma interna durante el arranque para descargar el modelo y dejarlo disponible antes de su uso. En el diagrama C.4 se puede observar como atiende el sistema las peticiones de los usuarios.

Arquitectura cliente-servidor multicapa

Desde el punto de vista lógico el sistema se estructura siguiendo una arquitectura cliente-servidor multicapa [5], organizada en tres niveles claramente diferenciados. Estos niveles se corresponden con la capa de presentación, la capa de aplicación y la capa de persistencia y servicios externos (ver arquitectura en la imagen C.5).

- **Capa de presentación:** es la interfaz de usuario (IU) del sistema y es accesible a través del navegador web. Se sigue un enfoque SSR

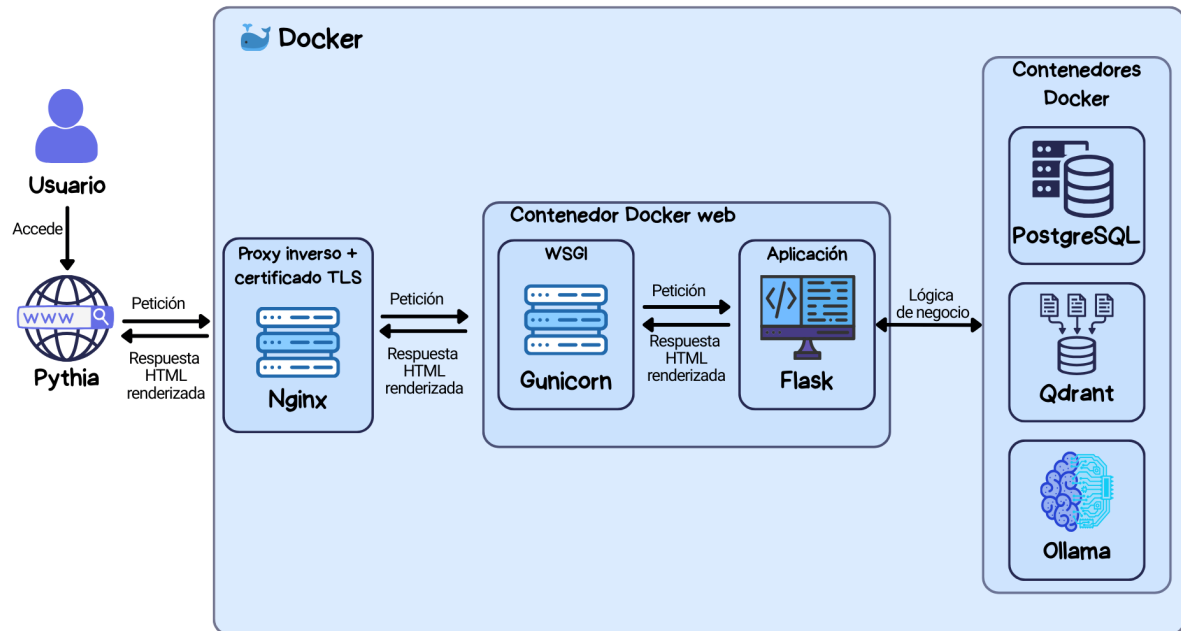


Figura C.4: Diagrama de la gestión de peticiones del sistema.

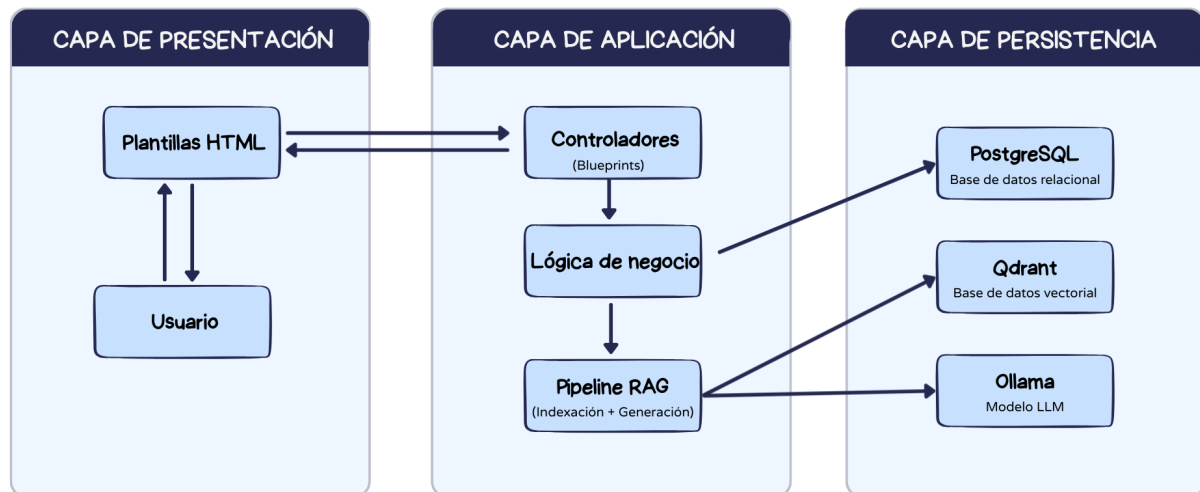


Figura C.5: Arquitectura cliente-servidor multicapa.

(*Server-Side Rendering*), en el que las páginas HTML se generan en el servidor mediante plantillas (usando las plantillas `Jinja2` en *Flask*) y se envían al navegador (cliente) ya renderizadas. Esto reduce la complejidad en el cliente, simplifica el control de acceso y la validación en el servidor. Mejora la trazabilidad y el registro de las interacciones y simplifica la integración del *pipeline RAG*.

- **Capa de aplicación:** se implementa en el contenedor *web*, que es donde se ejecuta la aplicación *Flask* servida por *Gunicorn*. Esta capa es responsable de coordinar la lógica funcional del sistema y actúa como núcleo del procesamiento. Esta compuesta por los controladores (rutas) organizados en *Blueprints* encargados de gestionar las peticiones HTTP, la lógica de negocio (gestión de usuarios, documentos, consultas, etc) y el *pipeline RAG*.
- **Capa de persistencia y servicios externos:** en esta capa se agrupan los sistemas encargados del almacenamiento y procesamiento de los datos. Se incluyen las bases de datos, tanto la relacional *PostgreSQL* como la vectorial *Qdrant*. También, se incluye *Ollama* como proveedor del modelo *LLM*. Todos estos servicios se ejecutan como contenedores independientes dentro de la red interna de *Docker*, garantizando aislamiento, modularidad y facilidad de despliegue.

Arquitectura de la aplicación *web*

La aplicación *web* sigue una arquitectura MVC (*Model-View-Controller*). Esta arquitectura es típica de las aplicaciones *Flask Server-Side-Rendered* (SSR) [3]. La técnica SSR consiste en generar el código de las páginas en el lado del servidor y entregar al cliente el código HTML completamente montado con toda la información que se debe mostrar.

El modelo vista intención (MVC) [1] es un patrón arquitectónico de software que separa el código en tres capas independientes. La capa modelo donde se incluye la lógica y los datos, la capa vista donde se implementa la interfaz de usuario (*frontend*) y la capa controlador que actúa de intermediario entre las acciones del usuario en las vistas y la lógica de la aplicación (rutas). En esta aplicación se implementa adaptado al ecosistema *Flask* (ver diagrama C.6).

- La **vista:** está compuesta por las plantillas HTML renderizadas mediante el motor de plantillas `Jinja2`, responsables de presentar la interfaz y capturar las interacciones del usuario. Además, incorpora los recursos estáticos (`css` y `JavaScript`). Aunque, la aplicación incorpora *JavaS-*

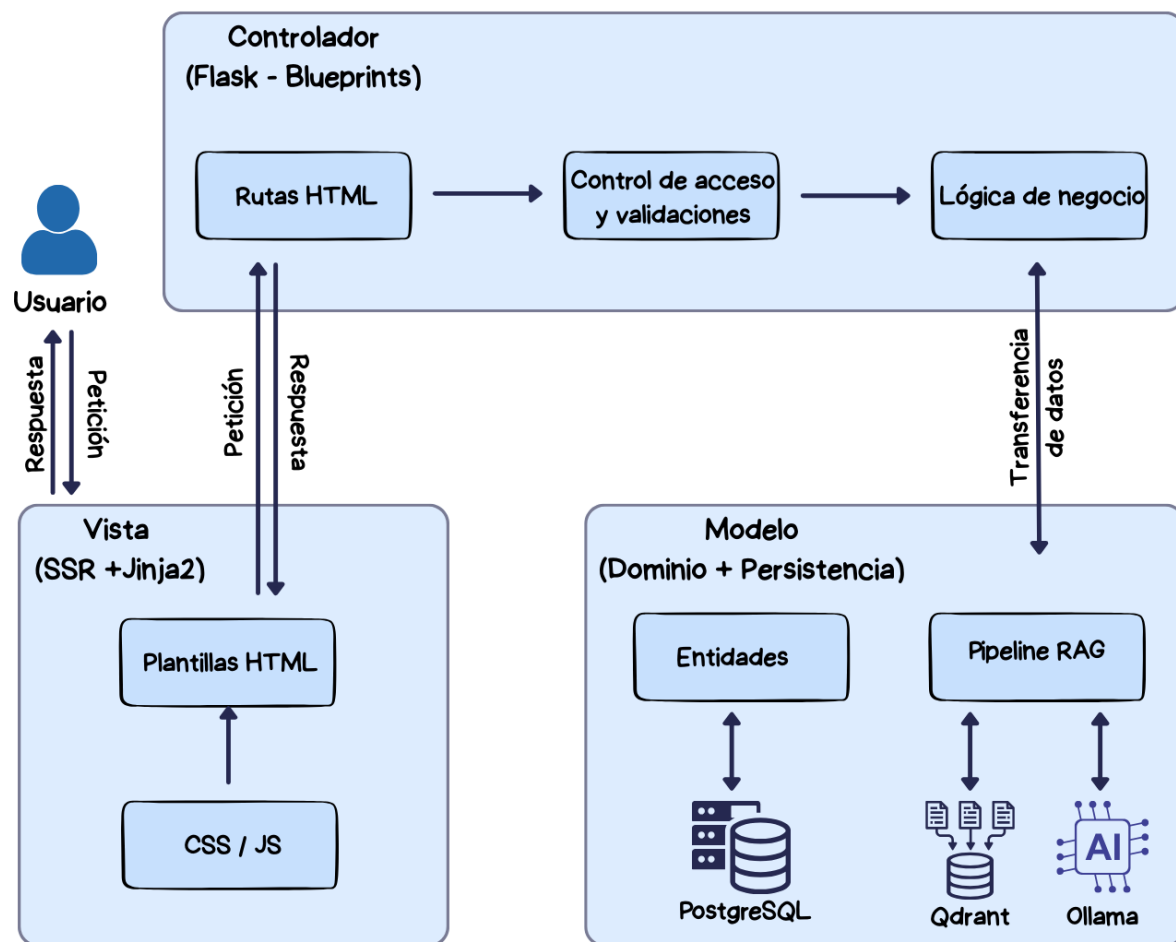


Figura C.6: Diagrama de la arquitectura Modelo-Vista-Controlador (MVC) del sistema.

cript para mejorar la experiencia de usuario, la generación principal del contenido es *server-side*, por lo que la arquitectura sigue siendo fundamentalmente SSR.

- El **controlador**: se implementa mediante rutas *Flask* organizadas en *Blueprints*. El uso de *Blueprints* permite mantener alta cohesión dentro de cada módulo y bajo acoplamiento entre funcionalidades. Se encarga de gestionar las peticiones HTTP, realizar validaciones básicas (como las de los formularios), así como controlar de acceso por rol. También, tiene la responsabilidad de invocar los servicios donde se encuentra la lógica, coordinar el *pipeline RAG* y generar las respuestas HTTP (renderizando la vista adecuada) .
- El **modelo**: esta compuesto por las entidades de datos persistentes (Usuario, Consulta, Documento, *Chunk*, *Embedding* y la relación *ConsultaChunk*) y por la lógica de negocio asociada (gestión de usuarios, administración documental, *pipelines* de consulta e indexación *RAG*, control de estados de procesos largos, etc). Se usa una base de datos relacional (*PostgreSQL*) para asegurar la persistencia de los datos de la aplicación y una base de datos vectorial (*Qdrant*) para la recuperación semántica. Esta capa permite desacoplar la lógica de negocio de la base de datos subyacente.

Con estas capas se mantiene el desacoplamiento de la aplicación *web*, haciendo que los controladores organicen el flujo para que la interfaz de usuario no implemente reglas de negocio y sea el modelo el que encapsule los datos y los procesos del dominio.

Arquitectura del sistema *RAG*

El sistema *RAG* se integra dentro de la capa de aplicación. Es el encargado de integrar en el sistema la generación de respuestas. u diseño se basa en la combinación de recuperación semántica sobre una base de datos vectorial y generación de texto mediante un modelo de lenguaje (*LLM*). A nivel de arquitectura se divide en dos *pipelines* [6], el de indexación y el de generación (ver diagrama C.7).

El ***pipeline* de indexación** se ejecuta cuando se cargan los documentos en la aplicación, ya bien sea de manera manual o mediante *web scraping*. Por cada documento se va a llevar a cabo una conversión a **Markdown** para que todos tengan una estructura similar que el *LLM* entienda fácilmente. De estos documentos se extrae, limpia y normaliza su texto, posteriormente se segmenta en *chunks* con solapamiento y se generan sus *embeddings*. Los

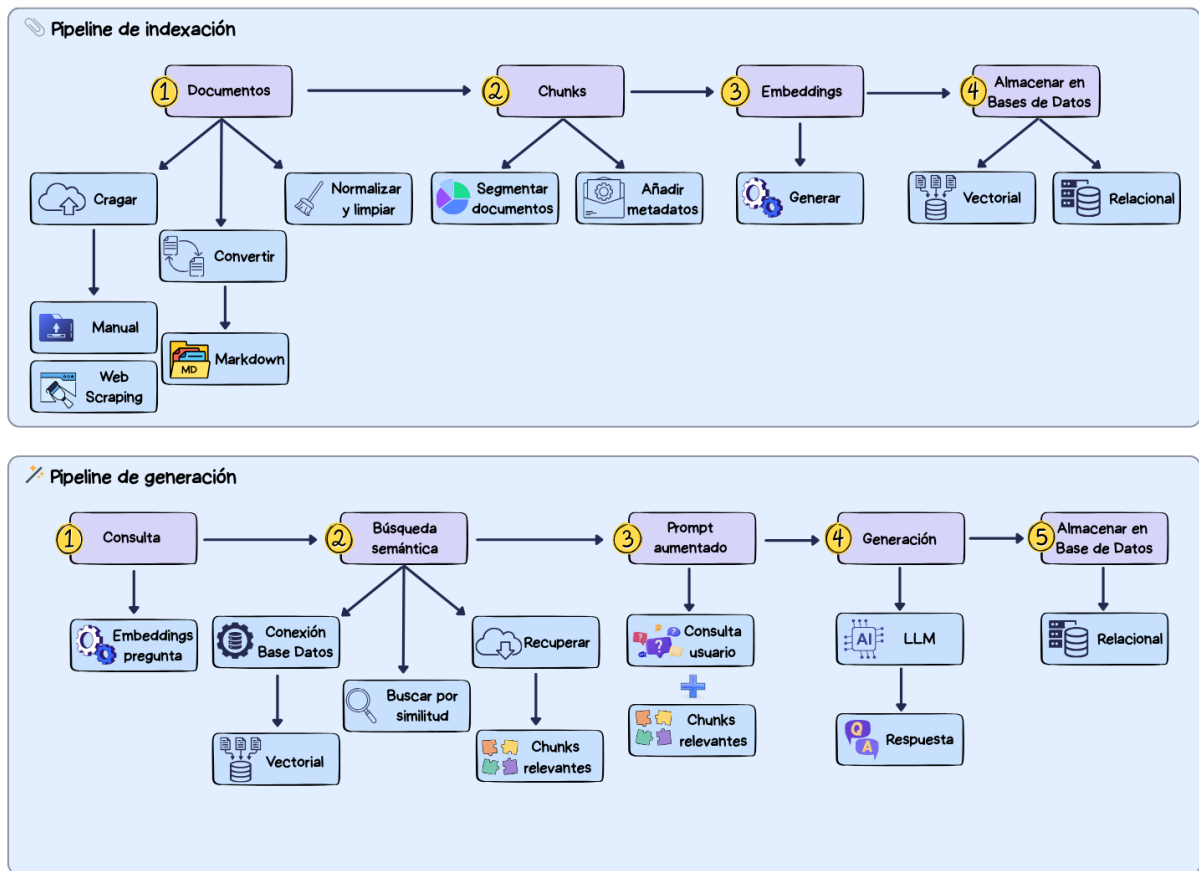


Figura C.7: Diagrama de la arquitectura del modelo *RAG*.

cuales se van a almacenar en la base de datos vectorial (*Qdrant*). Este proceso se ejecuta de forma asíncrona para evitar bloquear la interfaz *web*. Cuando acaba la indexación de los documentos se envía un correo electrónico al usuario para notificárselo.

El **pipeline de generación** se ejecuta cuando el usuario hace una consulta. Su objetivo es dar una respuesta contextualizada utilizando los *chunks* ya indexados en el sistema. Para ello va a generar los *embeddings* de la consulta y a recuperar los *chunks* más relevantes para generar la respuesta de *Qdrant*. Los *chunks* más relevantes se calculan por la similitud de sus *embeddings* con los de la consulta. Los usa para construir el *prompt* aumentado, el cual envía al modelo *LLM* para que responda a la consulta. Finalmente, muestra su respuesta al usuario y registra la consulta en la

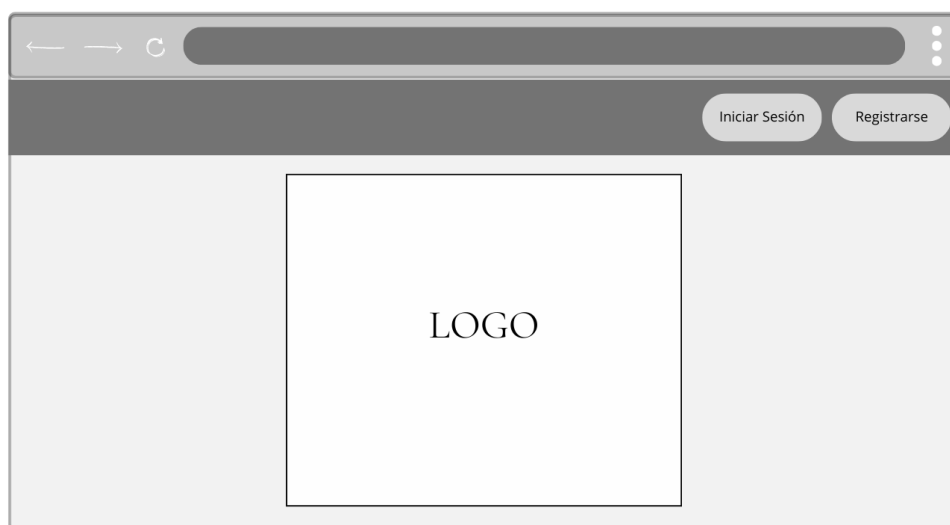


Figura C.8: Prototipo de la página de inicio.

base de datos relacional. Este *pipeline* combina recuperación semántica y generación de texto, asegurando trazabilidad al devolver los metadatos de los documentos fuente.

C.4. Diseño procedimental

C.5. Diseño de interfaces

El diseño de las interfaces se ha desarrollado de manera incremental. Inicialmente se hicieron bocetos genéricos para plasmar las principales funcionalidades de la aplicación. Posteriormente se fue iterando sobre esos diseños hasta llegar a las interfaces finales de la *web*.

Los prototipos iniciales que se crearon se ilustran en las figuras [C.8](#), [C.9](#), [C.10](#), [C.11](#), [C.12](#), [C.13](#), [C.14](#), [C.15](#), [C.16](#), [C.17](#) y [C.18](#).

Aunque estos prototipos se asemejan mucho a las interfaces finales, se han realizado diversos cambios. Por ejemplo, se ha cambiado la cabecera para aportar más información como el nombre del usuario que inicia sesión, su rol (usuario normal o administrador) y su último inicio de sesión (ver imagen de la cabecera final [C.19](#)). Además, se ha añadido a la tabla del historial de



Prototipo de la página de inicio de sesión. La interfaz muestra un navegador web con una barra de direcciones y botones de navegación. El contenido principal es un formulario centrado con el título "Inicio De Sesión". El formulario contiene dos campos de entrada: "Email" y "Contraseña", cada uno con un botón de "X" para borrar. Debajo de los campos hay dos botones: "Iniciar" y "Registrarse".

Figura C.9: Prototipo de la página de inicio de sesión.



Prototipo de la página de registro. La interfaz muestra un navegador web con una barra de direcciones y botones de navegación. El contenido principal es un formulario centrado con el título "Registro". El formulario contiene cuatro campos de entrada: "Nombre", "Email", "Contraseña" y "Repite la contraseña", cada uno con un botón de "X" para borrar. Debajo de los campos hay dos botones: "Registrarse" y "Iniciar Sesión".

Figura C.10: Prototipo de la página de registro.

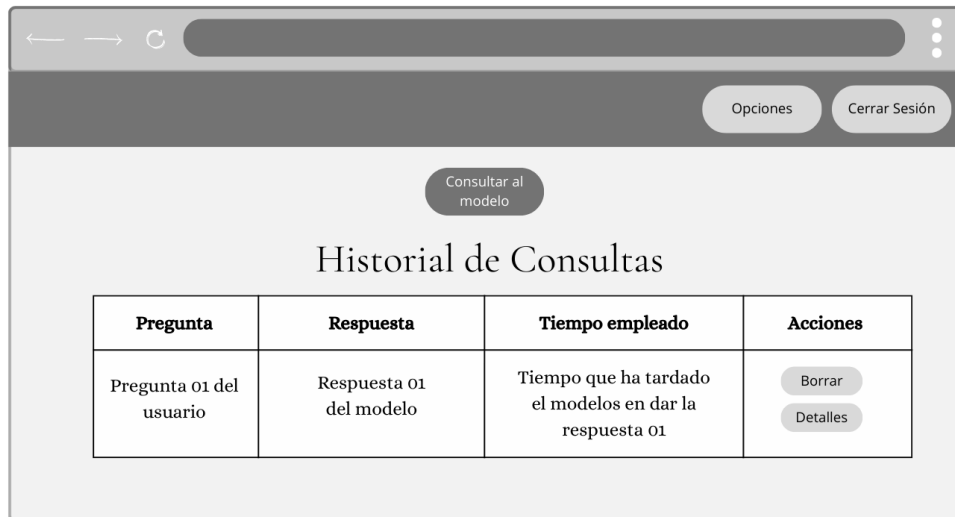


Figura C.11: Prototipo de la página principal.

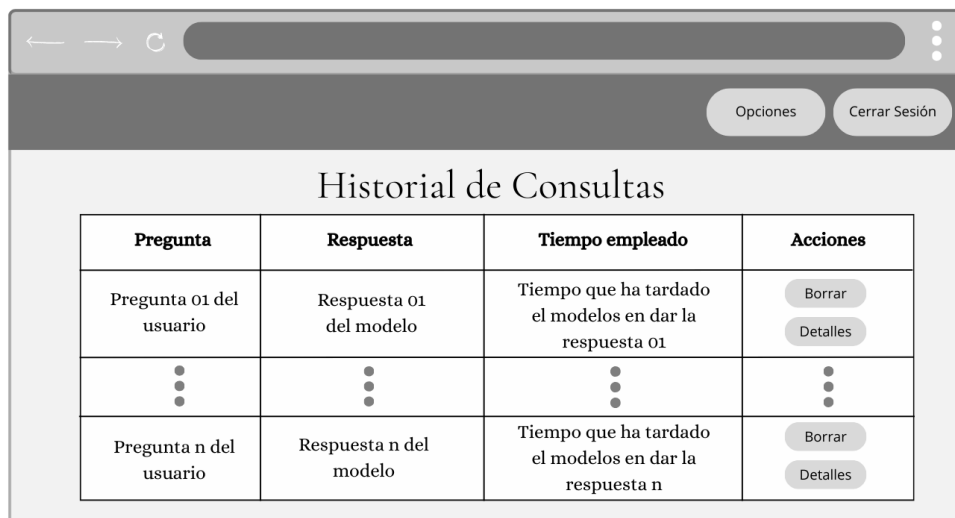


Figura C.12: Prototipo de la página del historial.

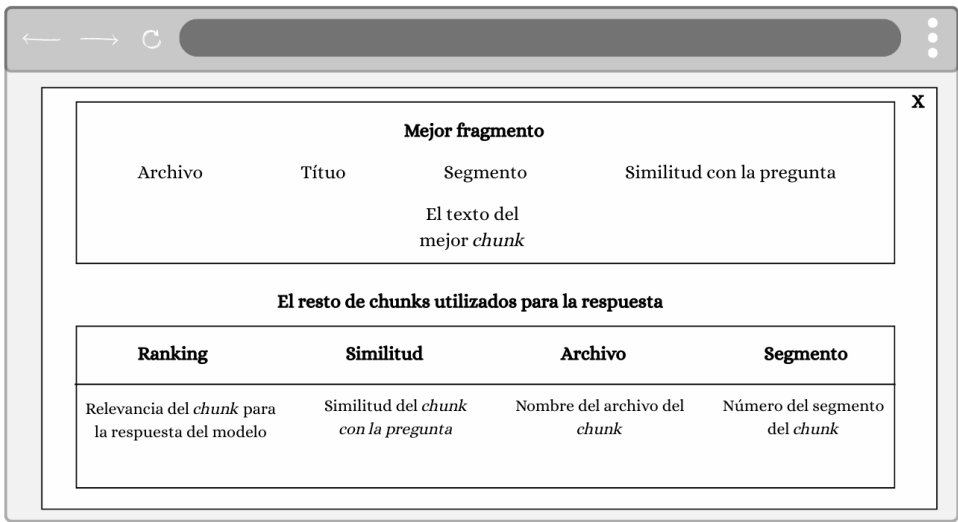


Figura C.13: Prototipo de los detalles del historial.

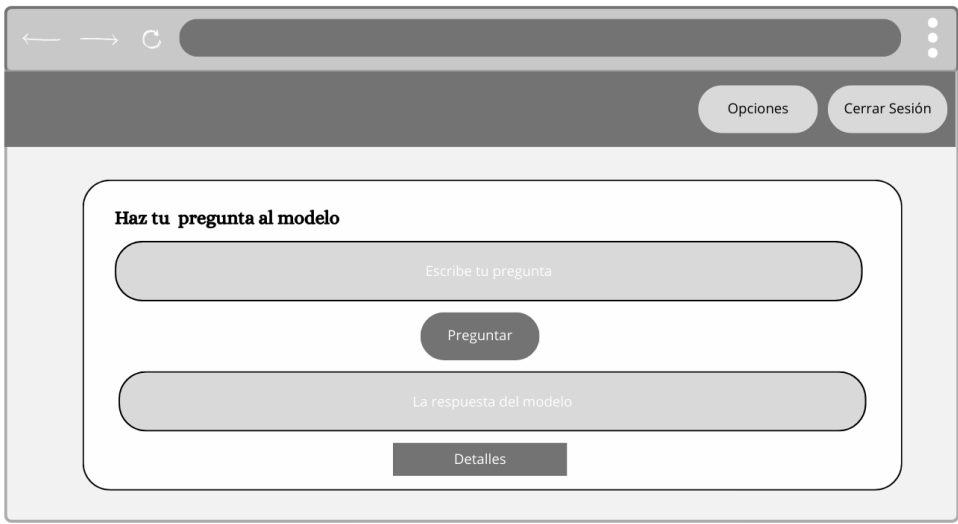
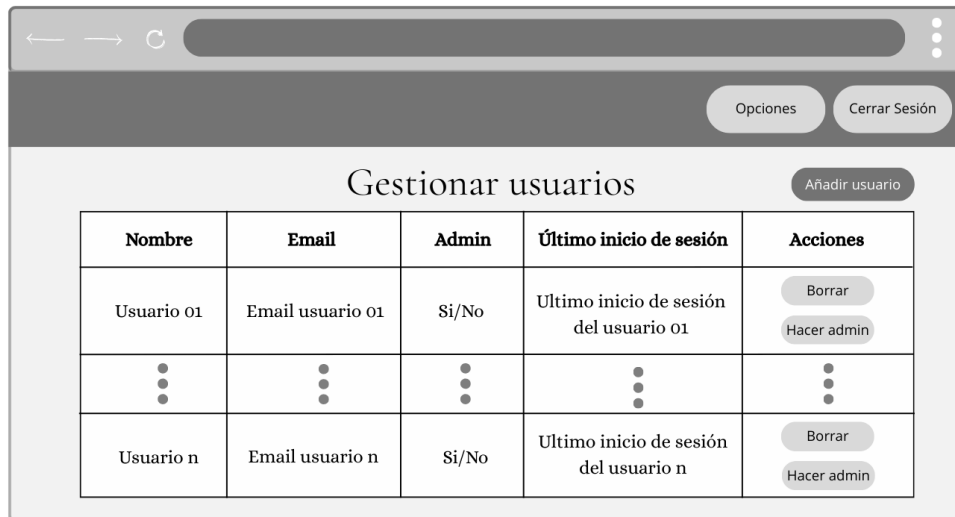


Figura C.14: Prototipo de la página de consulta.



Prototipo de la página de gestión de usuarios. La interfaz incluye una barra de navegación superior con botones "Opciones" y "Cerrar Sesión". El título principal es "Gestionar usuarios", acompañado de un botón "Añadir usuario". El contenido principal es una tabla con las siguientes columnas: Nombre, Email, Admin, Último inicio de sesión y Acciones. La tabla muestra dos usuarios: "Usuario 01" y "Usuario n".

Nombre	Email	Admin	Último inicio de sesión	Acciones
Usuario 01	Email usuario 01	Si/No	Ultimo inicio de sesión del usuario 01	<button>Borrar</button> <button>Hacer admin</button>
⋮	⋮	⋮	⋮	⋮
Usuario n	Email usuario n	Si/No	Ultimo inicio de sesión del usuario n	<button>Borrar</button> <button>Hacer admin</button>

Figura C.15: Prototipo de la página de gestión de usuarios.



Prototipo de la página de añadir usuarios. La interfaz muestra un formulario con los campos: Nombre, Email, Contraseña y un checkbox "¿Es administrador?". Un botón "Añadir usuario" está ubicado al final del formulario.

Añadir usuario

Nombre

Email

Contraseña

☐ ¿Es administrador?

Añadir usuario

Figura C.16: Prototipo de la página de añadir usuarios.



Figura C.17: Prototipo de la página de edición del usuario.

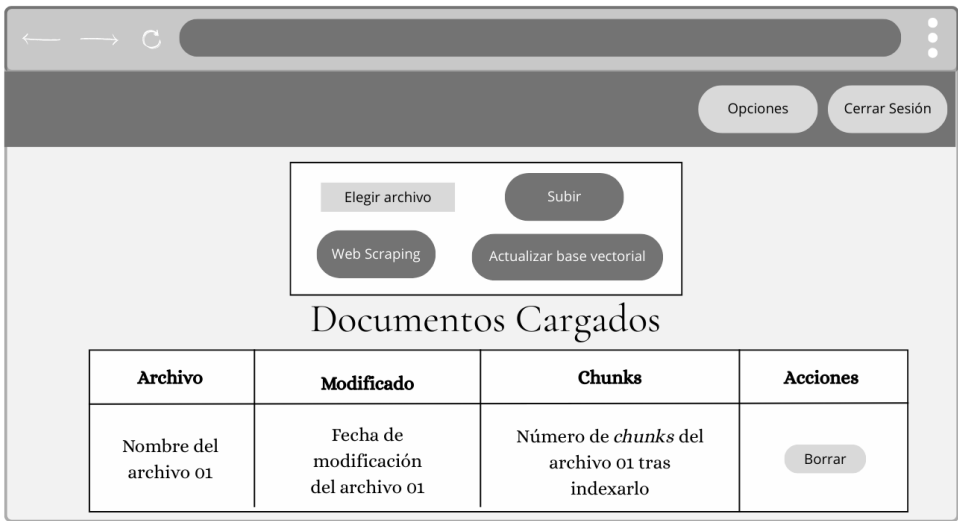


Figura C.18: Prototipo de la página de gestión de documentos.

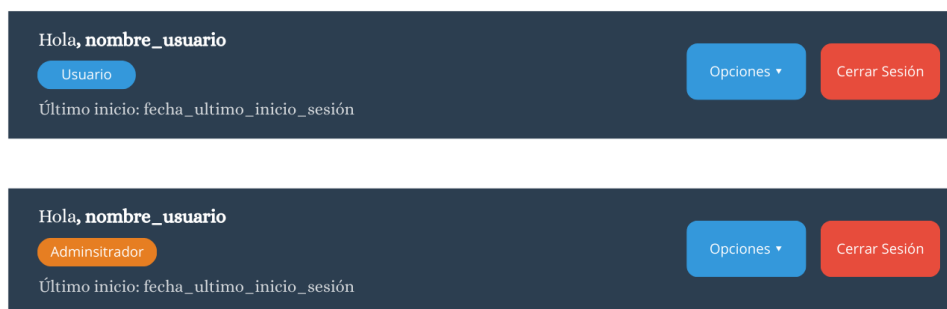


Figura C.19: Diseño de interfaz – Cabecera.

consultas y a la de documentos cargados información relevante que no se había valorado inicialmente. En la tabla del historial de consultas se han añadido diversos campos para aportar toda la información relevante sobre la consulta. Añadiendo el documento al que pertenece el *chunk* principal y la fecha en la que el usuario hizo la consulta. Para el administrador, también, se ha añadido una columna en la que se indica el nombre del usuario que hizo la consulta. Además, para mejorar la legibilidad de la tabla se ha decidido ubicar la respuesta debajo de la fila en un desplegable (ver imagen del historial de consulta definitivo C.20). Por su parte, en la tabla donde se muestran los documentos cargados del sistema se han añadido dos columnas, una que indica el tamaño del documento y otra que indica el estado de ese documento en el sistema (cargado, indexado o fallido) (ver imagen del diseño final de la tabla documentos C.21). Otro aspecto relevante es que en la página para realizar la consulta se ha añadido un desplegable para ver los *chunks* utilizados (ver imagen de la página de consultas definitiva C.22).

En estos diagramas C.23 y C.24 se muestra como sería la navegabilidad de los usuario por la aplicación dependiendo del rol que tienen (administrador o usuario normal).

Administrador:

Usuario	Fecha	Pregunta	Documento	Tiempo	Acciones
Nombre del usuario que hizo la consulta n	Fecha en la que el usuario hizo la consulta n	Pregunta del usuario	Nombre del documento al que pertenece el <i>chunk</i> principal	Tiempo que tardó el modelo en responder	Borrar Detalles
▶ Ver respuesta					
Nombre del usuario que hizo la consulta 01	Fecha en la que el usuario hizo la consulta 01	Pregunta del usuario	Nombre del documento al que pertenece el <i>chunk</i> principal	Tiempo que tardó el modelo en responder	Borrar Detalles
▶ Ver respuesta					

Usuario normal:

Fecha	Pregunta	Documento	Tiempo	Acciones
Fecha en la que el usuario hizo la consulta n	Pregunta del usuario	Nombre del documento al que pertenece el <i>chunk</i> principal	Tiempo que tardó el modelo en responder	Borrar Detalles
▶ Ver respuesta				
Fecha en la que el usuario hizo la consulta 01	Pregunta del usuario	Nombre del documento al que pertenece el <i>chunk</i> principal	Tiempo que tardó el modelo en responder	Borrar Detalles
▶ Ver respuesta				

Figura C.20: Diseño de interfaz – Historial de consultas.

Archivo	Tamaño	Modificado	Chunk	Estado	Acciones
Nombre del documento 01	Tamaño del documento 01	Fecha de modificación en el sistema	Número de <i>chunks</i> indexados en el sistema	cargado/indexado/ fallido	Borrar
Resto de filas con los nombres	Resto de filas con el tamaño	Resto de filas con las fechas de modificación	Resto de filas con los <i>chunks de los documentos</i>	Resto de fila con el estado (cargado/indexado/ fallido)	Borrar
Nombre del documento n	Tamaño del documento n	Fecha de modificación en el sistema	Número de chunks indexados en el sistema	cargado/indexado/ fallido	Borrar

Figura C.21: Diseño de interfaz – Tabla de documentos cargados.

Preguntar al modelo (RAG)

Escribe tu pregunta y mira la respuesta y el fragmento usado.

Pregunta

Escribe tu pregunta....

Preguntar

Respuesta

► Fragmento usado

Figura C.22: Diseño de interfaz – Interfaz de consulta al modelo.

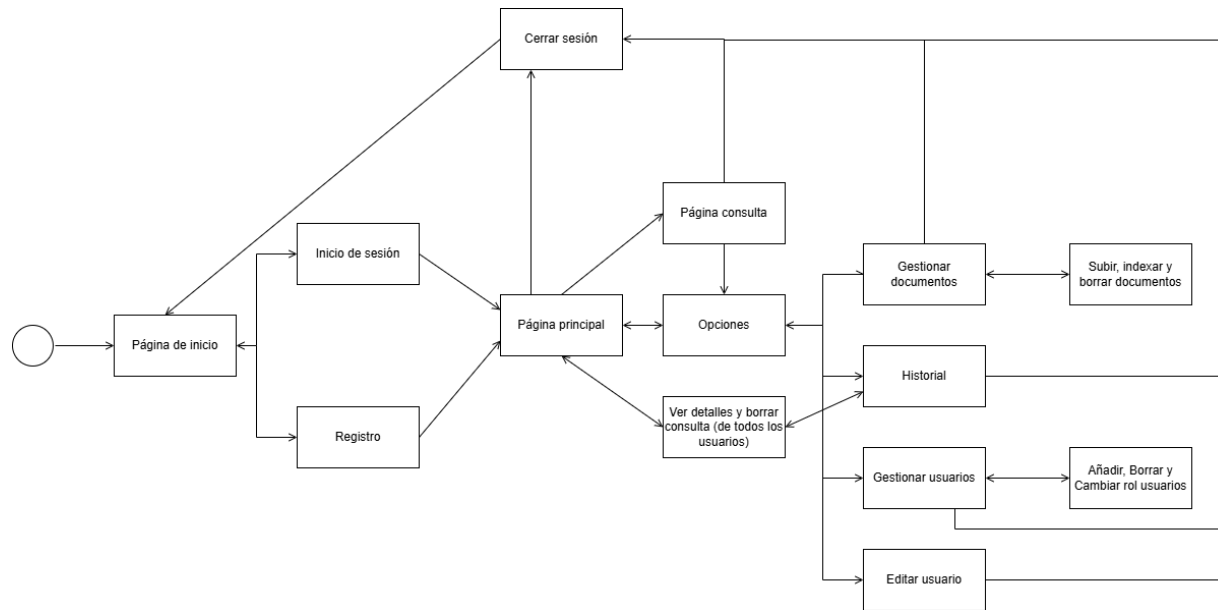


Figura C.23: Diagrama de navegabilidad por la aplicación para el administrador.

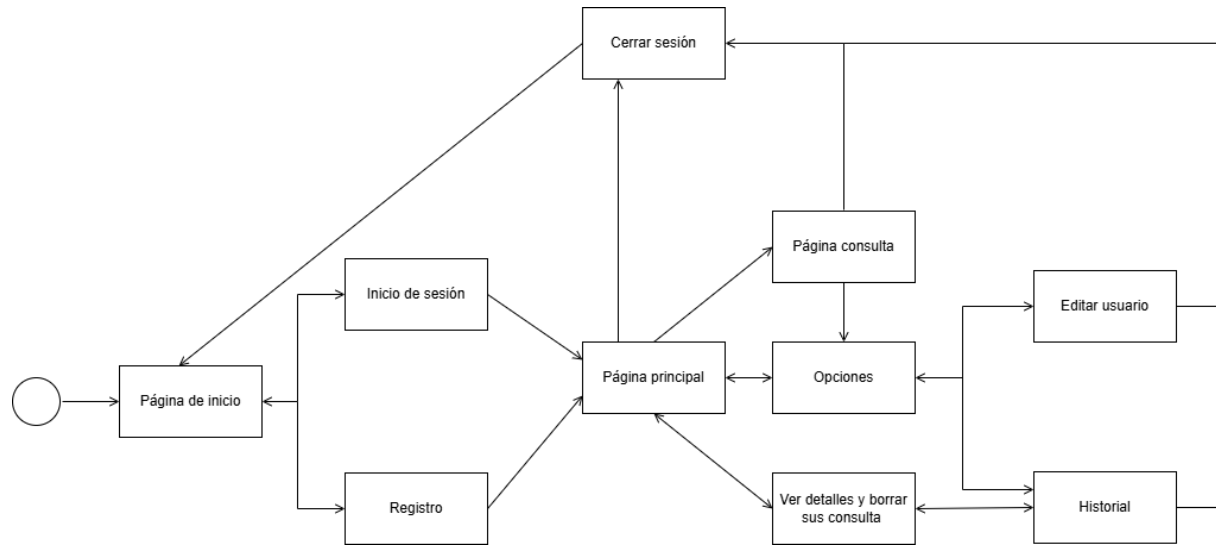


Figura C.24: Diagrama de navegabilidad por la aplicación para el usuario normal.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

D.2. Estructura de directorios

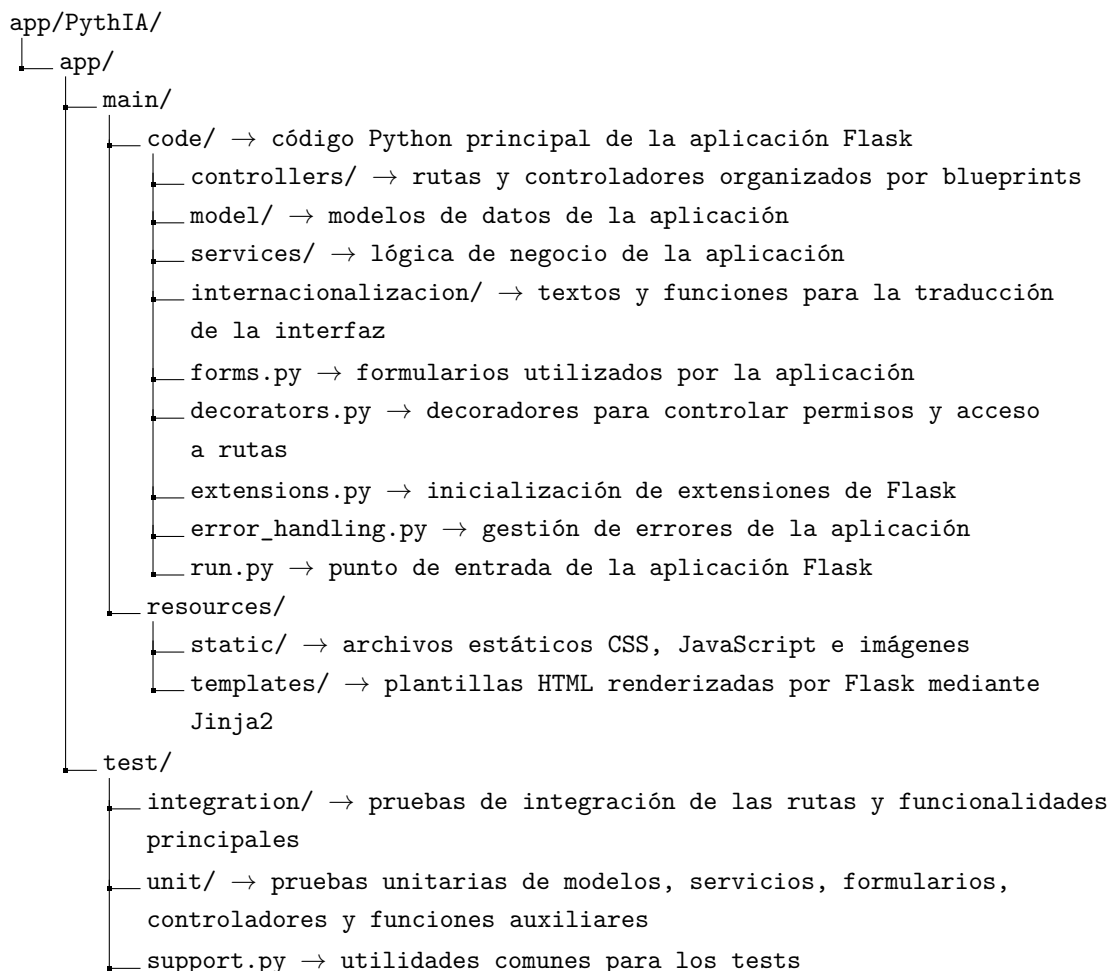
El proyecto se organiza en tres bloques principales, la aplicación final (PythIA), los prototipos desarrollados durante la fase experimental y la documentación del Trabajo de Fin de Grado.

```
/
├── .github/
│   └── workflows/
│       └── build.yml → flujo de integración continua del proyecto, ejecuta
│                       las comprobaciones automáticas definidas para GitHub Actions,
│                       incluyendo el análisis de calidad con SonarCloud
├── app/
│   └── PythIA/ → versión final de la aplicación web desarrollada con Flask
├── docs/
│   └── LaTeX/ → documentación técnica y memoria del proyecto en formato
│               LaTeX, junto con los documentos PDF generados
├── prototipos/ → carpeta con archivos de prueba desarrollados durante
│               el proyecto y versiones antiguas de la aplicación
└── .gitignore → archivo que indica qué ficheros o carpetas deben ser ignorados
                por Git y no subirse al repositorio
```

- sonar-project.properties → archivo de configuración para el análisis de calidad del código mediante SonarCloud
- README.md → archivo principal de documentación del proyecto, explica qué es PythIA y cómo desplegar la aplicación en local

El directorio `app/PythIA` contiene la versión final de la aplicación. En ella se encuentra la aplicación Flask organizada siguiendo una separación entre controladores, modelos, servicios, recursos estáticos y plantillas HTML. También incluye la configuración necesaria para ejecutar la aplicación en contenedores Docker, usando Gunicorn como servidor de aplicación y Nginx como *proxy* inverso.

Además, esta carpeta contiene los *tests* unitarios y de integración, que permiten comprobar el correcto funcionamiento de las rutas, modelos, servicios y funcionalidades principales. Las migraciones permiten mantener actualizada la estructura de la base de datos en los distintos entornos.



```

└─ run_tests.sh → script para ejecutar las pruebas
└─ docker/
    └─ entrypoint.sh → script de arranque del contenedor
    └─ requirements_docker.txt → dependencias Python necesarias para ejecutar
        la aplicación en Docker
    └─ Dockerfile → define la imagen Docker de la aplicación
    └─ Dockerfile.dockerignore → archivos excluidos al construir la imagen
        Docker
    └─ nginx/
        └─ nginx.conf.template → plantilla de configuración de Nginx como
            proxy inverso
└─ migrations/ → migraciones de la base de datos gestionadas con Flask-Migrate/Alembic
└─ .env → variables de entorno de la aplicación
└─ .gitattributes → configuración de atributos de Git
└─ .gitignore → archivos ignorados por Git dentro de la carpeta de la
    aplicación
└─ README.md → documentación específica de la aplicación final
└─ docker-compose.yml → define los servicios necesarios para ejecutar
    la aplicación

```

El directorio **controllers** contiene las rutas de Flask, organizadas mediante *Blueprints*. Esta organización permite dividir la aplicación en módulos funcionales independientes, manteniendo alta cohesión dentro de cada módulo y reduciendo el acoplamiento entre funcionalidades.

```

controllers/
└─ admin/
    └─ routes.py → registro, inicio de sesión, cierre de sesión y recuperación
        de contraseña
└─ auth/
    └─ routes.py → gestión de usuarios y documentos
└─ main/
    └─ routes.py → rutas generales de navegación, la página principal y
        el historial
└─ rag/
    └─ routes.py → interfaz de consulta al RAG y la gestión de respuestas

```

El directorio **model** contiene las entidades persistentes de la aplicación. Estas clases representan los datos almacenados en PostgreSQL. Permiten gestionar usuarios, documentos, consultas, *chunks*, *embeddings* y estados de procesos asíncronos.

```

model/

```

- `user.py` → define la entidad de usuarios del sistema (autenticación, roles y gestión de cuentas)
- `documento.py` → representa los documentos de licitaciones almacenados, incluyendo metadatos y estado de procesamiento
- `consulta.py` → modela las consultas realizadas por los usuarios al sistema RAG y sus resultados
- `chunk.py` → define los fragmentos de texto generados a partir de los documentos para su procesamiento
- `embedding.py` → representa los vectores generados a partir de los chunks para su almacenamiento en la base de datos vectorial
- `consulta_chunk.py` → modela la relación entre consultas y los chunks utilizados para generar la respuesta
- `job_state.py` → clase base para gestionar el estado de tareas largas o procesos en segundo plano
- `rag_query_state.py` → controla el estado de ejecución de las consultas al sistema RAG
- `web_scraping_state.py` → gestiona el estado de los procesos de web scraping
- `vector_update_state.py` → controla el estado de actualización de la base de datos vectorial
- `markdown_conversion_state.py` → gestiona el estado de los procesos de conversión de documentos a formato Markdown

El directorio `services` agrupa la lógica de negocio de la aplicación. A diferencia de los controladores, que reciben las peticiones HTTP, los servicios ejecutan las operaciones internas.

- `services/`
 - `async_tasks.py` → ejecución de procesos largos en segundo plano
 - `documentos.py` → gestión de documentos cargados en el sistema
 - `markdown/`
 - `Conversion_markdown.py` → implementa la conversión de documentos PDF a formato Markdown, incluyendo limpieza y estructuración del texto
 - `rag/`
 - `PrototipoRAGpy` → implementación del pipeline RAG (recuperación de chunks relevantes y generación de respuestas con el LLM)
 - `default_prompts.py` → define los prompts por defecto utilizados para interactuar con los modelos de lenguaje
 - `service.py` → orquesta el funcionamiento del sistema RAG integrando recuperación, generación y gestión de respuestas
 - `web_scraping/`

- DescargarPliegos.py → script encargado de descargar los documentos PDF de licitaciones
- PliegosPlaywrightAsincrono.py → implementa el web scraping asíncrono mediante Playwright para obtener datos de licitaciones
- pliegos_pdfs.json → almacena los enlaces y metadatos de los documentos PDF obtenidos
- resultados_playwright_asincrono_servidor.json → contiene los resultados del scraping (enlaces, información extraída, etc.)
- markdown_conversion_state.py → control de estado de las tareas de conversión a Markdown
- vector_update_state.py → control de estado de las tareas de actualización de la base de datos vectorial
- web_scraping_state.py → control de estado de las tareas del web scraping

En el directorio **resources** se encuentran los elementos de la capa de presentación de la aplicación. Incluye tanto las plantillas HTML renderizadas dinámicamente como los recursos estáticos utilizados por la interfaz web.

Dentro de él se encuentra el directorio **tempaltes** y el **static**. En el directorio **tempaltes** se encuentran las plantillas HTML renderizadas por Flask mediante el motor de plantillas Jinja2, las cuales forman la capa de vista de la aplicación. En el directorio **static** se encuentran los recursos estáticos que utiliza la aplicación web. Estos archivos no son procesados por el servidor y se envían directamente al navegador.

```
resources/
├── templates/
│   ├── base.html → plantilla base que define la estructura común (cabecera,
│   │             navegación, estilos), reutilizada por el resto mediante herencia
│   │             de Jinja2
│   ├── index.html → página de inicio pública de la aplicación
│   ├── login.html → interfaz de registro de usuarios
│   ├── singup.html → interfaz de registro de usuarios
│   ├── forgot_password.html → interfaz para solicitar que se envíe un correo
│   │                       cuando se ha olvidado la contraseña
│   ├── reset_password.html → interfaz para cambiar la contraseña cuando
│   │                       se olvida
│   ├── pag_principal.html → página principal tras autenticación
│   ├── rag.html → interfaz principal de consulta al sistema RAG
│   ├── rag_default_query.html → vista con consultas predefinidas
│   └── history.html → interfaz para ver el historial de consulta
```

- `tabla_historial.html` → plantilla con la tabla del historial para que la puedan reutilizar distintas interfaces
- `admin_documents.html` → gestión de documentos por parte del administrador
- `users.html` → interfaz para que el administrador pueda ver y gestionar los usuarios del sistema
- `admin_create_user.html` → página para que el administrador pueda registrar usuarios
- `edit_user.html` → platilla para que el usuario pueda editar su perfil
- `stats.html` → visualización de estadísticas del uso de los usuarios
- `model_comparison.html` → comparativa de modelos LLM cargados en el sistema
- `.3 error.html` → página de errores
- `static/`
 - `css/` → hojas de estilo utilizadas para definir el diseño visual de la aplicación
 - `js/` → scripts JavaScript que aportan interactividad a la interfaz (gestión de eventos, peticiones asíncronas, validaciones, gráficas, etc.).
 - `img/` → imágenes utilizadas en la interfaz

Esta separación de la aplicación permite tener tres módulos separados claramente para facilitar el mantenimiento de la interfaz y permitir cambios en el diseño sin afectar a la lógica del sistema.. El primero contiene la lógica de negocio (*backend*) definida en `controllers` y `services`, el segundo los datos (modelo) definida en `model` y el tercero la interfaz (vista) definida en `templates` y `static`.

La carpeta `prototipos` recoge las pruebas y versiones iniciales desarrolladas antes de llegar a la aplicación final. En ella se encuentran los primeros experimentos con RAG, la creación de la base de datos vectorial, la consulta mediante terminal, la descarga automática de pliegos, la conversión de documentos PDF a Markdown y las primeras versiones de la aplicación web.

- `prototipos/`
 - `BaseDatos.py` → script para crear la base de datos vectorial en Qdrant a partir de los documentos PDF
 - `PrototipoRAG.py` → primer prototipo completo del sistema RAG, con carga de documentos, generación de embeddings, búsqueda semántica y generación de respuestas con Ollama
 - `Prompt.py` → interfaz por terminal para realizar preguntas al prototipo RAG
 - `PruebaBaseDatos.py` → script para comprobar qué documentos y fragmentos han sido almacenados en la base de datos vectorial

- Flask/ → primer prototipo de la aplicación web Flask, con rutas, login, administración y uso de SQLite
- Flask_docker/ → versión posterior del prototipo Flask preparada para ejecutarse con Docker, Gunicorn, Nginx, PostgreSQL y Qdrant
- Markdown/ → pruebas de conversión de documentos PDF a Markdown mediante distintas técnicas, incluyendo OCR y modelos locales
- Web Scraping/ → scripts para recopilar licitaciones y descargar pliegos desde la Plataforma de Contratación del Sector Público
- tokenizers/ → pruebas de tokenización, división de documentos largos y generación de resúmenes
- pliegos/ → documentos PDF utilizados como datos de prueba
- qdrant_data/ → datos generados por la base de datos vectorial Qdrant
- requirements.txt → dependencias generales de los prototipos
- pyproject.toml → configuración del proyecto para Poetry
- poetry.lock → bloqueo de versiones de dependencias
- poetry.toml → configuración local de Poetry
- .gitignore → archivos ignorados dentro de la carpeta de prototipos

D.3. Manual del programador

Este apartado tiene como objetivo proporcionar a futuros desarrolladores que necesiten desplegar, modificar o ampliar la aplicación.

El proyecto implementa una aplicación basada en una arquitectura RAG (*Retrieval Augmented Generation*). Orientada a la gestión y consulta inteligente de documentación, utilizando tecnologías de procesamiento de lenguaje natural, recuperación semántica y generación de respuestas mediante modelos de inteligencia artificial.

El sistema ha sido desarrollado principalmente en Python utilizando el framework Flask para la capa web y diferentes librerías relacionadas con embeddings, recuperación documental y modelos de lenguaje. Además, se emplea Docker para facilitar la ejecución del entorno y garantizar la reproducibilidad del sistema.

El desarrollo del proyecto se ha realizado utilizando las siguientes herramientas y tecnologías:

- Lenguaje de programación: Python 3.
- Framework *backend*: Flask.

- Base de datos relacional: SQLite/MySQL según entorno.
- Sistema de migraciones: Flask-Migrate.
- Sistema de plantillas: Jinja2.
- Gestión de dependencias: pip.
- Contenedores: Docker y Docker Compose.
- Control de versiones: Git y GitHub.
- Entorno de desarrollo: Visual Studio Code.
- Librerías de IA y recuperación semántica: LangChain, modelos de embeddings y Qdrant.

La estructura general del proyecto se encuentra organizada siguiendo una arquitectura modular que separa la lógica de negocio, las rutas, los formularios y las funcionalidades relacionadas con el sistema RAG. Concretamente la aplicación sigue una arquitectura cliente-servidor multicapa. La capa de presentación está formada por las plantillas HTML renderizadas mediante Flask y Jinja2, mientras que la lógica de negocio se implementa en los controladores y servicios internos. El componente principal del proyecto es el sistema RAG. Este sistema combina recuperación semántica de información y generación de respuestas utilizando modelos de lenguaje.

El núcleo del sistema se basa en un flujo RAG compuesto por las siguientes etapas:

1. Carga y procesamiento de documentos.
2. Conversión de documentos a formato Markdown.
3. Fragmentación del contenido textual.
4. Generación de *embeddings* semánticos.
5. Indexación de fragmentos en la base vectorial.
6. Recuperación de contexto relevante ante una consulta.
7. Generación de respuesta mediante un modelo de lenguaje.

Por otra parte, la aplicación implementa un sistema de Blueprints de Flask para dividir las distintas áreas funcionales del sistema.

1. Módulo de autenticación (**auth**): es el encargado de registrar usuarios, el inicio y cierre de sesión, la recuperación de contraseña, la gestión de sesiones y la validación de credenciales.
2. Módulo principal (**main**): se encarga de la página principal, así como de la navegación general. Gestiona el historial de las consultas y permite la visualización de la información del usuario.
3. Módulo RAG (**rag**): contiene la lógica para subir documentos, convertirlos a Markdown, procesar semánticamente, recuperar el contexto para la consulta del usuario y generar las respuestas llamando a un LLM.
4. Módulo de administración (**admin**): permite gestionar a los usuarios, asignarles permisos y administrar el sistema (carga de documentos, actualización de la base de datos vectorial, etc).

El flujo de datos del procesamiento documental tras la carga de un documento por parte de un usuario en el sistema es:

- Validación del archivo.
- Conversión del documento a texto Markdown.
- Limpieza y normalización del contenido.
- División en fragmentos de tamaño controlado.
- Generación de *embeddings* para cada fragmento.
- Almacenamiento de *embeddings* en la base vectorial.

El sistema mantiene la trazabilidad entre los fragmentos indexados y el documento original.

El flujo de datos de la recuperación semántica ante una consulta del usuario (ver imagen [D.1](#)):

- Se genera un *embedding* de la pregunta.
- Se realiza una búsqueda vectorial.
- Se recuperan los fragmentos más relevantes.

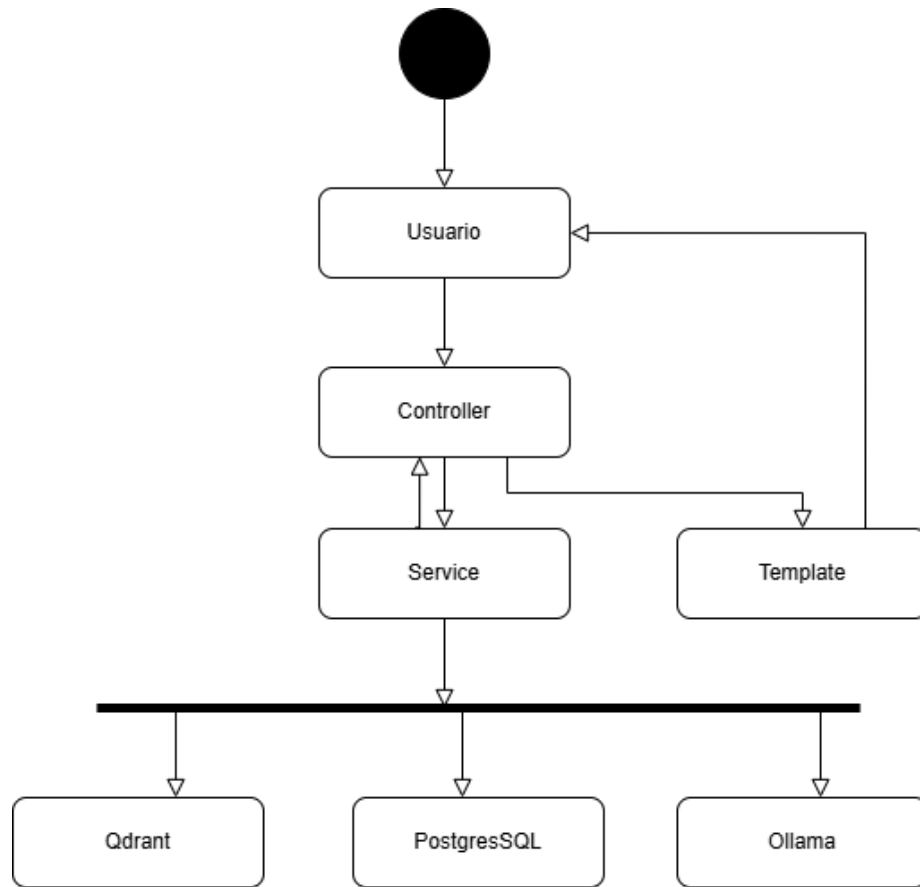


Figura D.1: Diagrama de flujo de la recuperación semántica tras una petición al sistema.

- El contexto recuperado se envía al modelo de lenguaje.
- El modelo genera una respuesta contextualizada.

Este enfoque permite reducir alucinaciones y mejorar la precisión de las respuestas generadas.

El sistema utiliza una base de datos relacional para almacenar:

- Usuarios.
- Roles.
- Historial de consultas.

- Metadatos de documentos.
- Información de indexación.

Las migraciones de la base de datos se gestionan mediante Flask-Migrate y Alembic.

Para crear una nueva migración se utiliza:

```
flask db migrate -m "descripcion"
```

Y para aplicarla:

```
flask db upgrade
```

La persistencia vectorial se realiza mediante Qdrant, donde se almacenan los *embeddings* generados durante la indexación.

El proyecto utiliza un amplio conjunto de dependencias organizadas por módulos funcionales. Estas librerías permiten implementar la aplicación web, la lógica del sistema RAG, la gestión documental, el procesamiento semántico y el despliegue completo del entorno.

Las dependencias del proyecto se encuentran definidas principalmente en el archivo `requirements.txt`. Para instalar todas las dependencias necesarias:

```
pip install -r requirements.txt
```

El núcleo de la aplicación se ha desarrollado utilizando Flask y diferentes extensiones asociadas.

- Flask: *framework* principal utilizado para construir la aplicación web y gestionar rutas, peticiones HTTP y renderizado de plantillas.
- Jinja2: motor de plantillas utilizado por Flask para generar páginas HTML dinámicas.
- Werkzeug: conjunto de utilidades internas de Flask relacionadas con seguridad, subida de archivos, gestión de peticiones y servidor WSGI.
- itsdangerous: librería utilizada para la generación de *tokens* seguros empleados en validaciones y recuperación de contraseñas.
- Flask-Login: extensión encargada de gestionar sesiones de usuario, autenticación y control de usuarios autenticados.

- Flask-WTF: integración de WTForms con Flask para facilitar la gestión de formularios.
- WTForms: librería para definición y validación de formularios web.
- Flask-Mail: extensión utilizada para el envío de correos electrónicos desde la aplicación.
- email-validator: validación de direcciones de correo electrónico introducidas por el usuario.
- python-dotenv: carga automática de variables de entorno definidas en archivos `.env`.
- gunicorn: servidor WSGI utilizado para ejecutar Flask en entornos de producción.

El *backend* utiliza tecnologías ORM, para mapear las estructuras de la base de datos relacional, y herramientas de migración para facilitar el acceso y gestión de la base de datos.

- SQLAlchemy: ORM utilizado para trabajar con la base de datos mediante clases Python.
- Flask-SQLAlchemy: integración de SQLAlchemy dentro del ecosistema Flask.
- Flask-Migrate: extensión que integra migraciones de base de datos con Flask.
- psycopg2: conector de Python para PostgreSQL.
- PostgreSQL: sistema gestor de base de datos relacional utilizado en producción.
- SQLite: base de datos ligera utilizada durante pruebas.
- pydantic: validación y estructuración tipada de datos.
- pydantic-core: motor interno optimizado utilizado por Pydantic.
- typing_extensions y typing-inspection: librerías auxiliares para anotaciones avanzadas de tipos.
- coverage: herramienta utilizada para medir la cobertura de los tests.

- unittest: framework de pruebas unitarias incluido en la librería estándar de Python.

La interfaz web se desarrolla mediante tecnologías estándar integradas con Flask.

- HTML: estructura principal de las páginas web.
- CSS: estilos visuales de la aplicación.
- JavaScript: implementación de interactividad en cliente.
- Jinja2: renderizado dinámico de contenido HTML.
- Static files de Flask: mecanismo utilizado para servir archivos CSS, JavaScript, imágenes, favicon y recursos estáticos.
- Plantillas Flask: páginas HTML dinámicas.

El sistema incorpora funcionalidades de extracción automática de documentación (*web Scraping*) desde páginas web de licitaciones.

- playwright: automatización de navegadores web para navegar y descargar documentación automáticamente.
- aiofiles: lectura y escritura asíncrona de archivos.
- asyncio: ejecución concurrente de tareas asíncronas.
- requests: realización de peticiones HTTP tradicionales.
- httpx: cliente HTTP moderno compatible con operaciones síncronas y asíncronas.
- urllib3: capa de comunicación HTTP utilizada internamente por otras librerías.
- certifi: certificados raíz para conexiones HTTPS seguras.
- idna: soporte para dominios internacionalizados.
- charset-normalizer: detección y normalización de codificaciones de texto.

El proyecto incorpora herramientas específicas para el tratamiento documental (procesamiento de PDF, Markdown y OCR).

- pypdf: extracción de texto desde documentos PDF.
- pdf2image: conversión de páginas PDF a imágenes.
- poppler-utils: herramientas del sistema necesarias para procesar PDF e imágenes.
- Pillow (PIL): procesamiento y manipulación de imágenes.
- base64: codificación de imágenes y archivos para intercambio con servicios externos.
- Nanonets OCR: sistema OCR utilizado para extraer texto desde documentos escaneados.
- Ollama: ejecución local de modelos OCR y modelos de lenguaje configurados en el despliegue.

El sistema RAG constituye el núcleo funcional del proyecto y requiere herramientas de procesamiento semántico y recuperación vectorial.

- qdrant-client: cliente Python utilizado para comunicarse con la base vectorial Qdrant.
- Qdrant: base de datos vectorial utilizada para almacenar *embeddings* y realizar búsquedas semánticas.
- sentence-transformers: generación de *embeddings* semánticos.
- transformers: utilización de modelos de Hugging Face.
- torch: *backend* de *deep learning* utilizado por los modelos de IA.
- huggingface-hub: descarga y gestión de modelos alojados en Hugging Face.
- tokenizers: tokenización eficiente de texto para modelos de lenguaje.
- safetensors: formato seguro y eficiente para almacenamiento de pesos de modelos.

- scikit-learn: utilidades de machine learning empleadas en diferentes procesos auxiliares.
- numpy y scipy: librerías de cálculo numérico y procesamiento matricial.
- Ollama: ejecución local de modelos LLM para generación de respuestas.

El entorno de ejecución completo del sistema se encuentra desacoplado mediante contenedores Docker.

- Docker: plataforma utilizada para empaquetar la aplicación y sus dependencias en contenedores.
- Docker Compose: orquestación de múltiples servicios del sistema.
- Nginx: proxy inverso encargado de gestionar conexiones HTTP y redirigir peticiones hacia Gunicorn.
- Gunicorn: servidor WSGI utilizado para ejecutar Flask en producción.
- PostgreSQL Docker image: contenedor de la base de datos relacional.
- Qdrant Docker image: contenedor de la base de datos vectorial.
- Ollama Docker image: contenedor encargado de ejecutar modelos LLM locales.
- Playwright Python Docker image: imagen base utilizada para las tareas de *webs craping* automatizado.

La configuración del sistema se realiza mediante variables de entorno, permitiendo desacoplar los parámetros sensibles y específicos del entorno de ejecución respecto al código fuente de la aplicación. Esto permite separar la configuración y lógica de negocio, adaptar fácilmente el sistema a distintos entornos, evitar exponer credenciales privadas en el repositorio, simplificar el despliegue mediante **Docker Compose** y mantener configuraciones reutilizables entre desarrollo y producción. En el proyecto se utilizan dos archivos de configuración el `.env` y el `secret.env`.

El archivo `.env` contiene variables de configuración general del sistema que no comprometen la seguridad de la aplicación, por lo cual, este archivo sí se incluye en el repositorio. Esto facilita la reproducción del entorno. En este archivo se definen las siguiente variables:

.env

```
FLASK_APP=app.main.code:create_app

POSTGRES_USER=postgres
POSTGRES_DB=mydatabase
POSTGRES_HOST=db
POSTGRES_PORT=5432
POSTGRES_EXTERNAL_PORT=5432

QDRANT_SERVICE_HOST=qdrant
QDRANT_INTERNAL_PORT=6333
QDRANT_URL_SCHEME=http

OLLAMA_IMAGE=ollama/ollama:0.20.7
OLLAMA_SERVICE_HOST=ollama
OLLAMA_INTERNAL_PORT=11434 OLLAMA_EXTERNAL_PORT=11435
OLLAMA_NUM_GPU=-1
RAG_LLM_MODEL=llama3.1:8b-instruct-q4_K_M
RAG_LLM_MODELS=llama3.1:8b-instruct-q4_K_M,gemma3:4b,qwen3:4b-instruct
OCR_MODEL_NAME=blaiifa/Nanonets-OCR-s
OLLAMA_PULL_TIMEOUT_SECONDS=1800
OLLAMA_PULL_IDLE_TIMEOUT_SECONDS=300
OLLAMA_PULL_LOG_INTERVAL_SECONDS=20

WEB_INTERNAL_PORT=5000
NGINX_HTTP_INTERNAL_PORT=80
NGINX_HTTP_EXTERNAL_PORT=7000
NGINX_HTTPS_INTERNAL_PORT=443
NGINX_HTTPS_EXTERNAL_PORT=443

DOCS_DIR=/data/pliegos

PROFILE_UPLOAD_FOLDER =/data/profiles

HF_HOME=/data/hf

QDRANT_COLLECTION=chunks
```

Estas variables permiten configurar el nombre de la aplicación Flask, el entorno de ejecución, los parámetros generales de la base de datos, la

dirección de los servicios Docker, la configuración de Qdrant, la configuración de Ollama, el puerto y las rutas internas.

El archivo `secret.env` contiene información sensible y credenciales privadas necesarias para el funcionamiento del sistema, por lo cual este archivo no se sube al repositorio GitHub del proyecto. esta separación de variables se ha realizado para mejorar la seguridad del sistema. Las variables que se definen en este archivo son:

`secret.env`

```
SECRET_KEY
FLASK_SESSION_SIGNER
POSTGRES_PASSWORD

MAIL_SERVER
MAIL_PORT
MAIL_USE_TLS
MAIL_USE_SSL
MAIL_USERNAME
MAIL_PASSWORD
MAIL_DEFAULT_SENDER
```

Estas variables sirven para definir la clave secreta de Flask, la contraseña de Postgres, las credenciales SMTP, los *tokens* de las API externas y las claves privadas utilizadas por los servicios OCR y modelos externos.

Las variables de entorno definidas en ambos archivos se cargan automáticamente en la aplicación durante el arranque. esto permite tener la configuración desacoplada del código fuente.

Importar variables de entorno

```
from dotenv import load_dotenv
load_dotenv()
```

Recuperar variables de entorno

```
import os
SECRET_KEY = os.getenv("SECRET_KEY")
```

Estas variables también se utilizan en el archivo `docker-compose.yml` para que cada contenedor reciba automáticamente las variables necesarias durante su inicialización.

docker-compose.yml

```
env_file:  
- .env  
- secret.env
```

El proyecto incorpora configuración Docker para facilitar el despliegue y la ejecución homogénea del entorno. Esto permite aislar la aplicación Flask, la base de datos relacional (postgres), la base de datos vectorial (qdrant) y las dependencias externas. El uso de docker simplifica la portabilidad y evita problemas derivados de diferencias entre sistemas operativos o versiones de dependencias. Para levantar el entorno hay que ejecutar el siguiente comando:

Terminal

```
docker-compose up -build
```

D.4. Compilación, instalación y ejecución del proyecto

En este apartado se describen los pasos necesarios para compilar, instalar y ejecutar el proyecto PythIA. El proyecto ha sido diseñado para ejecutarse principalmente mediante contenedores Docker, lo que permite reproducir el entorno completo de forma homogénea y transparente al usuario, independientemente del sistema operativo utilizado. La aplicación integra múltiples servicios que gracias a la utilización de Docker Compose se pueden desplegar de manera coordinada. Por ejemplo, la aplicación Flask, la base de datos PostgreSQL, la base de datos vectorial Qdrant, los modelos LLM ejecutados mediante Ollama y el servidor Nginx utilizado como proxy inverso.

Antes de ejecutar el sistema es necesario tener instalados Docker, Docker Compose, Git, Python (este último solo para ejecutarlo sin usar Docker). Debido al uso de modelos LLM y de OCR se recomienda que la aplicación se despliegue en una máquina que tenga procesador multinúcleo con al menos 16GB de memoria RAM y espacio suficiente para cargar los modelos y almacenar los documentos. Es recomendable que tenga GPU para acelerar los procesos.

Para instalar Docker en Ubuntu se deben ejecutar los siguientes comandos:

Terminal

```
sudo apt install ca-certificates curl gnupg lsb-release -y
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin -y
# Una vez instalado, se puede comprobar que Docker funciona correctamente
usando:
docker -version
```

Para instalar Docker en Windows se puede instalar descargando Docker Desktop desde la [página oficial](#)¹.

Terminal

```
# Una vez instalado, se puede comprobar que Docker funciona correctamente
usando:
docker -version
```

Para instalar Git en Ubuntu se usa el comando:

Terminal

```
sudo apt install git -y
```

En Windows se puede descargar desde la siguiente [página](#)².

En ambos casos para comprobar la instalación se usa:

Terminal

```
git -version
```

El código fuente del proyecto se encuentra disponible en el repositorio [GitHub](#)³ del proyecto. Por lo cual, el primer paso para la ejecución es clonar el repositorio desde GitHub.

Terminal

```
git clone https://github.com/lbr1014/TFG_RAG.git
```

Posteriormente se debe acceder al directorio del proyecto.

¹<https://www.docker.com/products/docker-desktop/>

²<https://git-scm.com/>

³https://github.com/lbr1014/TFG_RAG

Terminal

```
cd TFG_RAG/app/PythIA
```

La aplicación utiliza variables de entorno para almacenar información sensible y parámetros de configuración, como claves secretas, credenciales de correo, URLs de conexión o modelos LLM configurados.

Estas variables se definen mediante archivos `.env` y `secret.env`. El archivo `secret.env` no se incluye en el repositorio debido a que contiene información privada y credenciales sensibles, pero debe generarse para ejecutarlo, las variables que deben configurarse en dicho archivo son:

secret.env

```
SECRET_KEY
FLASK_SESSION_SIGNER
POSTGRES_PASSWORD

MAIL_SERVER
MAIL_PORT
MAIL_USE_TLS
MAIL_USE_SSL
MAIL_USERNAME
MAIL_PASSWORD
MAIL_DEFAULT_SENDER
```

Una vez instalado Docker y Git, descargado el repositorio y generado el archivo `secret.env` se puede ejecutar la aplicación. Para ello deben generarse las imágenes Docker. Esta construcción es automática gracias a los archivos `docker-compose.yml` y `Dockerfile`.

Terminal

```
# Para construir todas las imágenes y levantar el sistema completo
por primera vez se debe ejecutar:
docker compose up -build

# Si se desea reconstruir las imágenes sin reutilizar la caché
de Docker se puede usar:
docker compose build --no-cache

# Para reconstruir únicamente las imágenes sin ejecutar los
```

```
contenedores:  
docker compose build
```

Estos comandos van a construir las imágenes Docker, descargando las dependencias necesarias, levantando PostgreSQL, Qdrant, Ollama, inicializando la aplicación Flask y configurando Nginx y Gunicorn.

D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía

- [1] Miguel Angel Alvarez. Qué es mvc, 2023. [Online; descargado 1-Marzo-2026].
- [2] Cloudflare. ¿qué es tls (transport layer security)?, 2026. [Online; descargado 1-Marzo-2026].
- [3] Fernán García de Zúñiga. ¿qué es server side rendering (ssr) y cuáles son sus ventajas?, 2025. [Online; descargado 1-Marzo-2026].
- [4] Red Hat. Soa: ¿qué es la arquitectura orientada a los servicios?, 2026. [Online; descargado 1-Marzo-2026].
- [5] IBM. ¿qué es la arquitectura de tres niveles?, 2026. [Online; descargado 1-Marzo-2026].
- [6] Abhinav Kimothi. *A Simple Guide to Retrieval Augmented Generation*. Manning Publications, July 2025. [Consultado 8-Septiembre-2025 a 26-Septiembre-2025].
- [7] Kenneth C. Laudon and Jane P. Laudon. *Management Information Systems*. Pearson, 2012. [Consultado 15-Enero-2026 a 6-Febrero-2026].